

PY16CASINO

PY-16 FANTASY CONSOLE CARTRIDGE



BY PROF PLANLOZ

2026-05-24

OPEN THIS PDF WITH PY-16 TO PLAY

PY16CASINO - MANUAL

DESCRIPTION

=== ENGLISH ===

Welcome to py-16 Casino! A 5-in-1 experience with dynamic day/night cycles at the harbor. Build your bankroll!

1. SLOTS: Classic 5-reel slot. 3+ matching symbols from left to right win. High values for Diamonds (DIA) and Spades (SPA).
2. ROULETTE: Place chips strategically on single numbers (0-36), colors (Red/Black), Even/Odd, or Columns/Dozens.
3. BLACKJACK: Duel the dealer! Get as close to 21 as possible without busting. Face cards are 10, Aces 1 or 11.
4. VIDEO POKER: "Jacks or Better" 5-Card Draw. Choose which cards to "HOLD". Discards are replaced once.
5. PREMIUM (Oicho-Kabu): Traditional Japanese game. Only the last digit of the total counts (e.g., 15 = 5). 9 is the highest.

=== DEUTSCH ===

Willkommen im py-16 Casino! Ein 5-in-1 Erlebnis mit dynamischem Tag-/Nachtwechsel am Hafen. Baue deinen Bankroll auf!

1. SLOTS: 5-Walzen-Automat. 3+ gleiche Symbole von links nach rechts gewinnen. Wertvolle Symbole sind Diamanten (DIA) und Piketten (SPA).
2. ROULETTE: Setze Chips auf Zahlen (0-36), Farben (Rot/Schwarz), Gerade/Ungerade oder Kolonnen. Maximiere deine Chancen.
3. BLACKJACK: Schlage den Dealer! Komme so nah wie moeglich an 21, ohne dich zu ueberkaufen ("Bust").
4. VIDEO POKER: "Jacks or Better" 5-Card Draw. Waehle Karten zum Halten ("HOLD"). Der Rest wird neu gezogen.
5. PREMIUM (Oicho-Kabu): Japanisches Kartenspiel. Es zaehlt nur die letzte Ziffer der Summe (z.B. 15 = 5). 9 ist die Hoehste.

=== ESPAÑOL ===

¡Bienvenido a py-16 Casino! Una experiencia 5 en 1 con ciclos dinamicos de dia/noche en el puerto. ¡Aumenta tu bankroll!

1. SLOTS: Tragamonedas de 5 rodillos. 3 o mas simbolos iguales de izquierda a derecha ganan. ¡3+ simbolos!
2. ROULETTE: Apuesta a numeros (0-36), colores (Rojo/Negro), Par/Impar o docenas/columnas. Apuesta maxima.
3. BLACKJACK: ¡Vence al crupier! Acercate a 21 sin pasarte ("Bust"). Figuras valen 10, Ases 1 u 11. "Hit" para pedir mas.
4. VIDEO POKER: "Jacks or Better" de 5 cartas. Elige que cartas conservar ("HOLD"). Un par de Jotas o mejor gana.
5. PREMIUM (Oicho-Kabu): Juego tradicional japonés. Solo cuenta el ultimo dígito del total (ej. 15 = 5). 9 es el mas alto.

=== PORTUGUÊS (BRASIL) ===

Bem-vindo ao py-16 Casino! Uma experiência 5 em 1 com ciclo dinâmico de dia/noite no porto. Aumente seu bankroll!

1. SLOTS: Máquina clássica de 5 rolos. 3+ símbolos iguais da esquerda para a direita ganham. Diamantes (DIA) e Espadas (SPA).
2. ROULETTE: Aposte fichas em números (0-36), cores (Vermelho/Preto), Par/Ímpar ou dúzias/columnas. Aposte com estratégia.
3. BLACKJACK: Duele contra o dealer! Chegue o mais perto de 21 sem ultrapassar ("Bust"). Figuras valem 10.
4. VIDEO POKER: "Jacks or Better" de 5 cartas. Escolha quais cartas segurar ("HOLD"). O resto é trocado uma vez.
5. PREMIUM (Oicho-Kabu): Jogo tradicional japonês. Apenas o último dígito do total conta (ex. 15 = 5). 9 é o mais alto.

=== ROMANA ===

Bine ai venit la py-16 Casino! O experienta 5 in 1 cu cicluri dinamice zi/noapte in port. Creste-ti capitalul si castiga!

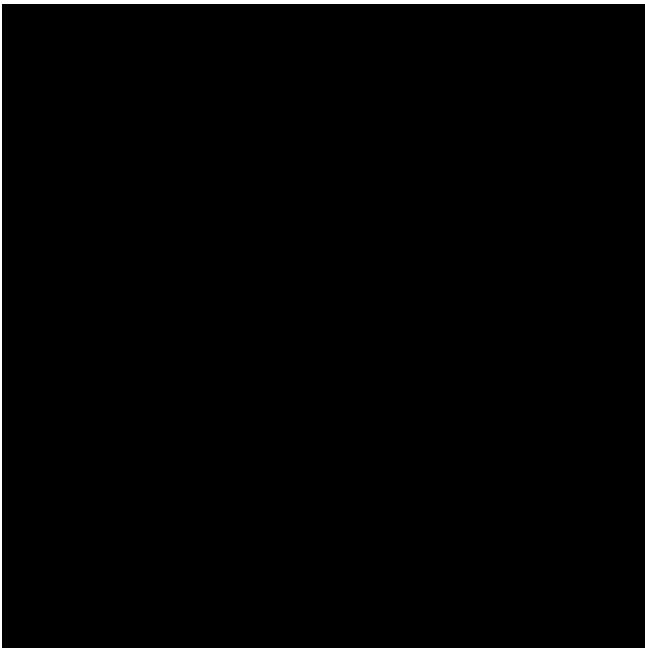
1. SLOTS: Slot clasic cu 5 role. 3+ simboluri identice de la stanga la dreapta castiga. Diamantele (DIA) si Septetele (SPA).
2. ROULETTE: Pune jetoane pe numere (0-36), culori (Rosu/Negru), Par/Impar sau coloane. Miza maxima este permisa.
3. BLACKJACK: Infrunta dealerul! Apropie-te cat mai mult de 21 fara sa depasesti ("Bust"). Figurile valoreaza 10.
4. VIDEO POKER: "Jacks or Better" cu 5 carti. Alege ce carti pastrezi ("HOLD"). Restul se schimba o data. O noua mana.
5. PREMIUM (Oicho-Kabu): Joc traditional japonez. Conteaza doar ultima cifra a sumei (ex. 15 = 5). 9 este cel mai mare.

CONTROLS

CREDITS

PY16CASINO - ASSETS

SPRITE SHEET (256x256)



MAP (128x128)



SFX SLOTS

MUSIC TRACKS

PY16CASINO - CODE LISTING (PAGE 1)

```
1 import py16
2 import random
3 import math
4 import datetime
5 import json
6 import os
7 from pathlib import Path
8
9 # @manual
10 # @description
11 # === ENGLISH ===
12 # Welcome to py-16 Casino! A 5-in-1 experience with dynamic day/night cycles at the harbor
13 #
14 # 1. SLOTS: Classic 5-reel slot. 3+ matching symbols from left to right win. High values f
15 # 2. ROULETTE: Place chips strategically on single numbers (0-36), colors (Red/Black), Eve
16 # 3. BLACKJACK: Duel the dealer! Get as close to 21 as possible without busting. Face card
17 # 4. VIDEO POKER: "Jacks or Better" 5-Card Draw. Choose which cards to "HOLD". Discards ar
18 # 5. PREMIUM (Oicho-Kabu): Traditional Japanese game. Only the last digit of the total cou
19 #
20 # === DEUTSCH ===
21 # Willkommen im py-16 Casino! Ein 5-in-1 Erlebnis mit dynamischem Tag-/Nachtwechsel am Haf
22 #
23 # 1. SLOTS: 5-Walzen-Automat. 3+ gleiche Symbole von links nach rechts gewinnen. Wertvolle
24 # 2. ROULETTE: Setze Chips auf Zahlen (0-36), Farben (Rot/Schwarz), Gerade/Ungerade oder K
25 # 3. BLACKJACK: Schlage den Dealer! Komme so nah wie moeglich an 21, ohne dich zu ueberkau
26 # 4. VIDEO POKER: "Jacks or Better" 5-Card Draw. Waehle Karten zum Halten ("HOLD"). Der Re
27 # 5. PREMIUM (Oicho-Kabu): Japanisches Kartenspiel. Es zaehlt nur die letzte Ziffer der Su
28 #
29 # === ESPAÑOL ===
30 # ¡Bienvenido a py-16 Casino! Una experiencia 5 en 1 con ciclos dinamicos de dia/noche en
31 #
32 # 1. SLOTS: Tragamonedas de 5 rodillos. 3 o mas simbolos iguales de izquierda a derecha ga
33 # 2. ROULETTE: Apuesta a numeros (0-36), colores (Rojo/Negro), Par/Impar o docenas/columna
34 # 3. BLACKJACK: ¡Vence al crupier! Acercate a 21 sin pasarte ("Bust"). Figuras valen 10, A
35 # 4. VIDEO POKER: "Jacks or Better" de 5 cartas. Elige que cartas conservar ("HOLD"). Un p
36 # 5. PREMIUM (Oicho-Kabu): Juego tradicional japonés. Solo cuenta el ultimo dígito del tot
37 #
38 # === PORTUGUÊS (BRASIL) ===
39 # Bem-vindo ao py-16 Casino! Uma experiência 5 em 1 com ciclo dinâmico de dia/noite no por
40 #
41 # 1. SLOTS: Máquina clássica de 5 rolos. 3+ símbolos iguais da esquerda para a direita gan
42 # 2. ROULETTE: Aposte fichas em números (0-36), cores (Vermelho/Preto), Par/Ímpar ou dúzia
43 # 3. BLACKJACK: Duele contra o dealer! Chegue o mais perto de 21 sem ultrapassar ("Bust").
44 # 4. VIDEO POKER: "Jacks or Better" de 5 cartas. Escolha quais cartas segurar ("HOLD"). O
45 # 5. PREMIUM (Oicho-Kabu): Jogo tradicional japonês. Apenas o último dígito do total conta
46 #
47 # === ROMANA ===
48 # Bine ai venit la py-16 Casino! O experienta 5 in 1 cu cicluri dinamice zi/noapte in port
49 #
50 # 1. SLOTS: Slot clasic cu 5 role. 3+ simboluri identice de la stanga la dreapta castiga.
51 # 2. ROULETTE: Pune jetoane pe numere (0-36), culori (Rosu/Negru), Par/Impar sau coloane.
52 # 3. BLACKJACK: Infrunta dealerul! Apropie-te cat mai mult de 21 fara sa depasesti ("Bust"
53 # 4. VIDEO POKER: "Jacks or Better" cu 5 carti. Alege ce carti pastrezi ("HOLD"). Restul s
54 # 5. PREMIUM (Oicho-Kabu): Joc traditional japonez. Conteaza doar ultima cifra a sumei (ex
55 #
56 # @controls
57 # [KEYBOARD] | [GAMEPAD] | [ACTION]
58 # Arrows | D-Pad / Stick | Navigate / Navigieren / Navegar / Navigare
59 # Z | A Button | Confirm / Bestaetigen / Confirmar / Confirma
60 # X | B Button | Special / Spezial / Especial / Special
61 # ENTER | Start Button | Menu / Menue / Menu / Meniu
62 # SPACE | Select / Back | Return / Zurueck / Volver / Voltar / Inapoi
63 #
64 # @credits
```

PY16CASINO - CODE LISTING (PAGE 2)

```
65 # Vibecode : Prof Planloz
66 # Code: Claud & Gemini
67 # Engine: py-16 Fantasy Console
68 # @end
69
70 # Game & Menu State
71 state = "MENU"
72 prev_state = "BETTING"
73 game_started = False
74 sound_enabled = True
75 menu_idx = 0
76
77 # Game Modes
78 modes = ["PREMIUM", "POKER", "ROULETTE", "BLACKJACK", "SLOTS"]
79 game_mode = "PREMIUM"
80
81 money = 500
82 jackpot = 10000
83
84 # Roulette-Verlauf: die letzten N Ergebnisse (neueste vorn).
85 # Persistent ueber Sessions, wie die Anzeigetafel an echten Roulette-Tischen.
86 roulette_history = []
87 ROULETTE_HISTORY_MAX = 15
88
89 # --- ACHIEVEMENTS ---
90 # Liste von (id, name, beschreibung). Reihenfolge bestimmt die Anzeige.
91 ACHIEVEMENTS = [
92     # Allgemein
93     ("FIRST_WIN", "First Win", "Win your first round"),
94     ("BIG_WIN", "Big Win", "Win 500+ in one round"),
95     ("HUGE_WIN", "Huge Win", "Win 2000+ in one round"),
96     ("MILLIONAIRE", "Millionaire", "Reach $10,000"),
97     ("BANKRUPT", "Rock Bottom", "Hit $0 and get reset"),
98     ("VETERAN", "Veteran", "Play 100 rounds"),
99     ("HIGH_ROLLER", "High Roller", "Place a $250 bet"),
100     # Blackjack
101     ("BJ_NATURAL", "Natural 21", "Hit a Blackjack (A + 10)"),
102     ("BJ_DOUBLE_WIN", "Doubling Down", "Win a Double Down"),
103     ("BJ_SPLIT", "Splitter", "Play a Split hand"),
104     ("BJ_DEALER_BUST", "House Crashed", "Make the dealer bust"),
105     # Roulette
106     ("ROUL_STRAIGHT", "Lucky Number", "Win a straight-up bet"),
107     ("ROUL_ZERO", "Le Zero", "Hit zero with a bet on it"),
108     # Slots
109     ("SLOT_FREE_SPINS", "Bonus Round", "Trigger free spins"),
110     ("SLOT_FIVE_KIND", "Maximum Reels", "Match 5 of a kind"),
111     # Premium
112     ("PREM_MINOR", "Minor Jackpot", "Hit the 3-8 jackpot"),
113     ("PREM_MAJOR", "Major Jackpot", "Hit the 3-9 jackpot"),
114     ("PREM_MAXIMUM", "Maximum Jackpot", "Hit the 8-9 jackpot"),
115     # Video Poker
116     ("POKER_FULL", "Full House", "Hit Full House in Poker"),
117     ("POKER_FOUR", "Four of a Kind", "Hit Four of a Kind"),
118     ("POKER_STRAIGHT_FLUSH", "Straight Flush", "Hit a Straight Flush"),
119     ("POKER_ROYAL", "Royal Crown", "Hit a Royal Flush"),
120 ]
121 ACH_TOTAL = len(ACHIEVEMENTS)
122 ACH_IDS = {a[0] for a in ACHIEVEMENTS}
123
124 unlocked_achievements = set()
125
126 # Cursor im Achievement-Listen-Screen (0..ACH_TOTAL-1)
127 ach_cursor = 0
128
```

PY16CASINO - CODE LISTING (PAGE 3)

```
129 # Toast-Queue: noch zu zeigende Achievements (Tupel a aus ACHIEVEMENTS)
130 achievement_toast_queue = []
131 achievement_toast_timer = 0 # 0 = kein Toast aktiv
132 ACH_TOAST_DURATION = 180 # Frames Gesamtdauer
133 ACH_TOAST_FADE = 20 # Frames Ein-/Ausblende
134
135 def unlock(ach_id):
136     """Schaltet ein Achievement frei und queued den Toast. No-op wenn schon offen."""
137     if ach_id not in ACH_IDS:
138         return
139     if ach_id in unlocked_achievements:
140         return
141     unlocked_achievements.add(ach_id)
142     # Achievement-Tupel finden und einreihen
143     for a in ACHIEVEMENTS:
144         if a[0] == ach_id:
145             achievement_toast_queue.append(a)
146             break
147
148 # --- Lifetime-Stats (persistent ueber Sessions) ---
149 lifetime_won = 0 # Summe aller Gewinne ueber alle Sessions
150 lifetime_lost = 0 # Summe aller Verluste ueber alle Sessions
151 games_played = 0 # Abgeschlossene Runden
152 biggest_win = 0 # Groesster Einzel-Gewinn
153
154 # Geld vor Rundenstart (fuer Delta-Berechnung beim GAME_OVER)
155 _money_at_round_start = None
156
157 # --- Persistenz (JSON unter ~/.py16/casino_save.json) ---
158 SAVE_VERSION = 1
159 try:
160     _SAVE_DIR = Path.home() / ".py16"
161     SAVE_PATH = _SAVE_DIR / "casino_save.json"
162 except Exception:
163     SAVE_PATH = None # Falls Home-Dir nicht verfuegbar
164
165 def save_state():
166     """Schreibt money/jackpot/Lifetime-Stats nach SAVE_PATH. Fehler werden geschluckt."""
167     if SAVE_PATH is None:
168         return False
169     data = {
170         "version": SAVE_VERSION,
171         "money": int(money),
172         "jackpot": int(jackpot),
173         "lifetime_won": int(lifetime_won),
174         "lifetime_lost": int(lifetime_lost),
175         "games_played": int(games_played),
176         "biggest_win": int(biggest_win),
177         "roulette_history": [int(x) for x in roulette_history],
178         "achievements": sorted(unlocked_achievements),
179         "difficulty": difficulty,
180         "language": language,
181     }
182     try:
183         _SAVE_DIR.mkdir(parents=True, exist_ok=True)
184         # Atomar schreiben: erst .tmp, dann replace
185         tmp = SAVE_PATH.with_suffix(".tmp")
186         with open(tmp, "w", encoding="utf-8") as f:
187             json.dump(data, f, indent=2)
188         os.replace(tmp, SAVE_PATH)
189         return True
190     except (OSError, IOError, TypeError):
191         return False
192
```

PY16CASINO - CODE LISTING (PAGE 4)

```
193 def load_state():
194     """Laedt Save-File und ueberschreibt die Globalen. Tolerant gegen Korruption."""
195     global money, jackpot, lifetime_won, lifetime_lost, games_played, biggest_win
196     global roulette_history, unlocked_achievements, difficulty, language
197     if SAVE_PATH is None or not SAVE_PATH.exists():
198         return False
199     try:
200         with open(SAVE_PATH, "r", encoding="utf-8") as f:
201             data = json.load(f)
202             if not isinstance(data, dict):
203                 return False
204             # Defensive Range-Checks: keine negativen Werte, kein absurdes Geld
205             m = int(data.get("money", money))
206             if 0 <= m <= 10**9: money = m
207             j = int(data.get("jackpot", jackpot))
208             if 0 <= j <= 10**9: jackpot = j
209             lw = int(data.get("lifetime_won", 0))
210             if lw >= 0: lifetime_won = lw
211             ll = int(data.get("lifetime_lost", 0))
212             if ll >= 0: lifetime_lost = ll
213             gp = int(data.get("games_played", 0))
214             if gp >= 0: games_played = gp
215             bw = int(data.get("biggest_win", 0))
216             if bw >= 0: biggest_win = bw
217             # Roulette-Historie: nur gueltige Zahlen 0..36, auf Max-Laenge gekuerzt
218             hist = data.get("roulette_history", [])
219             if isinstance(hist, list):
220                 cleaned = [int(x) for x in hist if isinstance(x, (int, float)) and 0 <= int(x)]
221                 roulette_history = cleaned[:ROULETTE_HISTORY_MAX]
222             # Achievements: nur bekannte IDs uebernehmen (Forward-Compat: alte Saves ohne Feld
223             ach = data.get("achievements", [])
224             if isinstance(ach, list):
225                 unlocked_achievements = {a for a in ach if isinstance(a, str) and a in ACH_IDS}
226             # Schwierigkeit: nur bekannte Werte uebernehmen
227             diff = data.get("difficulty", "MEDIUM")
228             if diff in ("EASY", "MEDIUM", "HARD"):
229                 difficulty = diff
230             # Sprache: nur bekannte IDs uebernehmen (Forward-Compat: alte Saves ohne Feld OK)
231             lang = data.get("language", "EN")
232             if lang in LANG_IDS:
233                 language = lang
234             return True
235     except (OSError, IOError, ValueError, json.JSONDecodeError, TypeError):
236         return False
237
238 # --- SCHWIERIGKEITSGRADE ---
239 # (id, anzeige-name, start-bankroll, beschreibung)
240 # WICHTIG: vor load_state() definieren, damit der Load 'difficulty' setzen kann.
241 DIFFICULTIES = [
242     ("EASY", "EASY", 10000, "Relaxed start. Big cushion."),
243     ("MEDIUM", "MEDIUM", 5000, "Balanced challenge."),
244     ("HARD", "HARD", 1000, "Small bankroll. High stakes."),
245 ]
246 difficulty = "MEDIUM" # wird beim Erststart ueber den Screen gesetzt
247 diff_cursor = 1 # Cursor auf dem Schwierigkeits-Screen (Default: MEDIUM)
248
249 # =====
250 # SPRACHSYSTEM (Menue + Screens; Spieltische bleiben Englisch)
251 # =====
252 # WICHTIG: vor load_state() definieren, damit der Load 'language' setzen kann.
253 # (id, anzeige-name in eigener Sprache)
254 LANGUAGES = [
255     ("EN", "ENGLISH"),
256     ("DE", "DEUTSCH"),
```

PY16CASINO - CODE LISTING (PAGE 5)

```
257     ("ES", "ESPAÑOL"),
258     ("PT", "PORTUGUES"),
259     ("RO", "ROMANA"),
260 ]
261 LANG_IDS = {l[0] for l in LANGUAGES}
262 language = "EN" # Default; wird per Save oder Menue gesetzt
263
264 # UI-Strings je Sprache. Schluesssel sind sprachneutral.
265 # EN ist die Referenz; fehlt ein Key in einer Sprache, faellt tr() auf EN zurueck.
266 TR = {
267     "EN": {
268         "menu_start": "START GAME", "menu_resume": "RESUME GAME", "menu_end": "END GAME",
269         "lbl_game": "GAME", "lbl_sound": "SOUND", "on": "ON", "off": "OFF",
270         "lbl_achievements": "ACHIEVEMENTS", "lbl_language": "LANGUAGE",
271         "menu_help": "SELECT: UP/DOWN  MODIFY: L/R",
272         "stat_games": "GAMES", "stat_net": "NET", "stat_biggest": "BIGGEST WIN",
273         "diff_welcome": "WELCOME TO THE CASINO", "diff_choose": "CHOOSE YOUR DIFFICULTY",
274         "diff_desc_easy": "Relaxed start. Big cushion.",
275         "diff_desc_medium": "Balanced challenge.",
276         "diff_desc_hard": "Small bankroll. High stakes.",
277         "diff_bankroll": "START BANKROLL",
278         "diff_help": "UP / DOWN: SELECT      Z: CONFIRM",
279         "diff_once": "THIS CHOICE IS MADE ONCE",
280         "ach_unlocked": "UNLOCKED", "ach_locked": "LOCKED",
281         "ach_help": "MOVE: ARROWS  BACK: X / ENTER",
282         "toast_unlocked": "ACHIEVEMENT UNLOCKED",
283     },
284     "DE": {
285         "menu_start": "SPIEL STARTEN", "menu_resume": "FORTSETZEN", "menu_end": "BEENDEN",
286         "lbl_game": "SPIEL", "lbl_sound": "TON", "on": "EIN", "off": "AUS",
287         "lbl_achievements": "ERFOLGE", "lbl_language": "SPRACHE",
288         "menu_help": "WAEHLEN: HOCH/RUNTER AENDERN: L/R",
289         "stat_games": "SPIELE", "stat_net": "NETTO", "stat_biggest": "BESTER GEWINN",
290         "diff_welcome": "WILLKOMMEN IM CASINO", "diff_choose": "WAEHLE DEINE STUFE",
291         "diff_desc_easy": "Entspannt. Grosses Polster.",
292         "diff_desc_medium": "Ausgewogene Herausforderung.",
293         "diff_desc_hard": "Wenig Geld. Hoher Einsatz.",
294         "diff_bankroll": "STARTKAPITAL",
295         "diff_help": "HOCH/RUNTER: WAEHLEN  Z: OK",
296         "diff_once": "DIESE WAHL GILT NUR EINMAL",
297         "ach_unlocked": "OFFEN", "ach_locked": "GESPERRT",
298         "ach_help": "BEWEGEN: PFEILE  ZURUECK: X",
299         "toast_unlocked": "ERFOLG FREIGESCHALTET",
300     },
301     "ES": {
302         "menu_start": "INICIAR JUEGO", "menu_resume": "CONTINUAR", "menu_end": "TERMINAR",
303         "lbl_game": "JUEGO", "lbl_sound": "SONIDO", "on": "ON", "off": "OFF",
304         "lbl_achievements": "LOGROS", "lbl_language": "IDIOMA",
305         "menu_help": "ELEGIR: ARRIBA/ABAJO CAMBIAR: L/R",
306         "stat_games": "JUEGOS", "stat_net": "NETO", "stat_biggest": "MAYOR PREMIO",
307         "diff_welcome": "BIENVENIDO AL CASINO", "diff_choose": "ELIGE LA DIFICULTAD",
308         "diff_desc_easy": "Inicio relajado. Buen colchon.",
309         "diff_desc_medium": "Desafio equilibrado.",
310         "diff_desc_hard": "Poco dinero. Alto riesgo.",
311         "diff_bankroll": "FONDOS INICIALES",
312         "diff_help": "ARRIBA/ABAJO: ELEGIR  Z: OK",
313         "diff_once": "ESTA ELECCION ES UNICA",
314         "ach_unlocked": "LOGRADO", "ach_locked": "BLOQUEADO",
315         "ach_help": "MOVER: FLECHAS  VOLVER: X",
316         "toast_unlocked": "LOGRO DESBLOQUEADO",
317     },
318     "PT": {
319         "menu_start": "INICIAR JOGO", "menu_resume": "CONTINUAR", "menu_end": "ENCERRAR",
320         "lbl_game": "JOGO", "lbl_sound": "SOM", "on": "LIG", "off": "DES",
```

PY16CASINO - CODE LISTING (PAGE 6)

```
321     "lbl_achievements": "CONQUISTAS", "lbl_language": "IDIOMA",
322     "menu_help": "ESCOLHER: CIMA/BAIXO  MUDAR: L/R",
323     "stat_games": "JOGOS", "stat_net": "SALDO", "stat_biggest": "MAIOR PREMIO",
324     "diff_welcome": "BEM-VINDO AO CASSINO", "diff_choose": "ESCOLHA A DIFICULDADE",
325     "diff_desc_easy": "Inicio tranquilo. Boa reserva.",
326     "diff_desc_medium": "Desafio equilibrado.",
327     "diff_desc_hard": "Pouco dinheiro. Alto risco.",
328     "diff_bankroll": "SALDO INICIAL",
329     "diff_help": "CIMA/BAIXO: ESCOLHER  Z: OK",
330     "diff_once": "ESTA ESCOLHA E UNICA",
331     "ach_unlocked": "LIBERADO", "ach_locked": "BLOQUEADO",
332     "ach_help": "MOVER: SETAS  VOLTAR: X",
333     "toast_unlocked": "CONQUISTA LIBERADA",
334 },
335 "RO": {
336     "menu_start": "INCEPE JOCUL", "menu_resume": "CONTINUA", "menu_end": "TERMINA",
337     "lbl_game": "JOC", "lbl_sound": "SUNET", "on": "PORNIT", "off": "OPRIT",
338     "lbl_achievements": "REALIZARI", "lbl_language": "LIMBA",
339     "menu_help": "SELECTARE: SUS/JOS  MODIFICA: L/R",
340     "stat_games": "JOCURI", "stat_net": "NET", "stat_biggest": "CASTIG MAXIM",
341     "diff_welcome": "BINE AI VENIT LA CAZINOU", "diff_choose": "ALEGE DIFICULTATEA",
342     "diff_desc_easy": "Start relaxat. Rezerva mare.",
343     "diff_desc_medium": "Provocare echilibrata.",
344     "diff_desc_hard": "Bani putini. Mize mari.",
345     "diff_bankroll": "CAPITAL INITIAL",
346     "diff_help": "SUS/JOS: ALEGE  Z: OK",
347     "diff_once": "ALEGEREA SE FACE O DATA",
348     "ach_unlocked": "DEBLOCAT", "ach_locked": "BLOCAT",
349     "ach_help": "MISCARE: SAGETI  INAPOI: X",
350     "toast_unlocked": "REALIZARE DEBLOCATA",
351 },
352 }
353
354 # Achievement-Uebersetzungen: ACH_TR[lang][ach_id] = (name, desc).
355 # EN kommt aus der ACHIEVEMENTS-Liste selbst (kein Eintrag hier noetig).
356 ACH_TR = {
357     "DE": {
358         "FIRST_WIN": ("Erster Sieg", "Gewinne deine erste Runde"),
359         "BIG_WIN": ("Grosser Gewinn", "Gewinne 500+ in einer Runde"),
360         "HUGE_WIN": ("Riesengewinn", "Gewinne 2000+ in einer Runde"),
361         "MILLIONAIRE": ("Millionaer", "Erreiche $10.000"),
362         "BANKRUPT": ("Pleite", "Auf $0 fallen und Reset"),
363         "VETERAN": ("Veteran", "Spiele 100 Runden"),
364         "HIGH_ROLLER": ("Hochroller", "Setze $250 auf einmal"),
365         "BJ_NATURAL": ("Natuerliche 21", "Erziele ein Blackjack (A+10)"),
366         "BJ_DOUBLE_WIN": ("Verdoppelt", "Gewinne ein Double Down"),
367         "BJ_SPLIT": ("Splitter", "Spiele eine geteilte Hand"),
368         "BJ_DEALER_BUST": ("Bank kaputt", "Bringe den Dealer zum Bust"),
369         "ROUL_STRAIGHT": ("Gluecks-Zahl", "Gewinne eine Zahlenwette"),
370         "ROUL_ZERO": ("Le Zero", "Triff die Null mit Einsatz"),
371         "SLOT_FREE_SPINS": ("Bonusrunde", "Loese Freispiele aus"),
372         "SLOT_FIVE_KIND": ("Volle Walzen", "Triff 5 Gleiche"),
373         "PREM_MINOR": ("Mini-Jackpot", "Knacke den 3-8 Jackpot"),
374         "PREM_MAJOR": ("Gross-Jackpot", "Knacke den 3-9 Jackpot"),
375         "PREM_MAXIMUM": ("Max-Jackpot", "Knacke den 8-9 Jackpot"),
376         "POKER_FULL": ("Full House", "Triff Full House im Poker"),
377         "POKER_FOUR": ("Vierling", "Triff einen Vierling"),
378         "POKER_STRAIGHT_FLUSH": ("Straight Flush", "Triff einen Straight Flush"),
379         "POKER_ROYAL": ("Royale Krone", "Triff einen Royal Flush"),
380     },
381     "ES": {
382         "FIRST_WIN": ("Primera Victoria", "Gana tu primera ronda"),
383         "BIG_WIN": ("Gran Premio", "Gana 500+ en una ronda"),
384         "HUGE_WIN": ("Premio Enorme", "Gana 2000+ en una ronda"),
```

PY16CASINO - CODE LISTING (PAGE 7)

```
385     "MILLIONAIRE": ("Millionario", "Alcanza $10.000"),
386     "BANKRUPT": ("En la Ruina", "Llega a $0 y reinicia"),
387     "VETERAN": ("Veterano", "Juega 100 rondas"),
388     "HIGH_ROLLER": ("Gran Apostador", "Apuesta $250 de una vez"),
389     "BJ_NATURAL": ("21 Natural", "Consigue un Blackjack (A+10)"),
390     "BJ_DOUBLE_WIN": ("Doblar y Ganar", "Gana un Double Down"),
391     "BJ_SPLIT": ("Divisor", "Juega una mano dividida"),
392     "BJ_DEALER_BUST": ("Banca Hundida", "Haz que la banca se pase"),
393     "ROUL_STRAIGHT": ("Numero de Suerte", "Gana una apuesta a numero"),
394     "ROUL_ZERO": ("Le Zero", "Acierta el cero apostado"),
395     "SLOT_FREE_SPINS": ("Ronda Bonus", "Activa giros gratis"),
396     "SLOT_FIVE_KIND": ("Rodillos Llenos", "Consigue 5 iguales"),
397     "PREM_MINOR": ("Jackpot Menor", "Gana el jackpot 3-8"),
398     "PREM_MAJOR": ("Jackpot Mayor", "Gana el jackpot 3-9"),
399     "PREM_MAXIMUM": ("Jackpot Maximo", "Gana el jackpot 8-9"),
400     "POKER_FULL": ("Full House", "Consigue Full House"),
401     "POKER_FOUR": ("Poker", "Consigue un Poker"),
402     "POKER_STRAIGHT_FLUSH": ("Escalera Color", "Consigue Escalera de Color"),
403     "POKER_ROYAL": ("Corona Real", "Consigue Escalera Real"),
404 },
405 "PT": {
406     "FIRST_WIN": ("Primeira Vitoria", "Ganhe sua primeira rodada"),
407     "BIG_WIN": ("Grande Premio", "Ganhe 500+ numa rodada"),
408     "HUGE_WIN": ("Premio Enorme", "Ganhe 2000+ numa rodada"),
409     "MILLIONAIRE": ("Milionario", "Alcance $10.000"),
410     "BANKRUPT": ("Na Pior", "Chegue a $0 e reinicie"),
411     "VETERAN": ("Veterano", "Jogue 100 rodadas"),
412     "HIGH_ROLLER": ("Grande Apostador", "Aposte $250 de uma vez"),
413     "BJ_NATURAL": ("21 Natural", "Consiga um Blackjack (A+10)"),
414     "BJ_DOUBLE_WIN": ("Dobrou e Ganhou", "Ganhe um Double Down"),
415     "BJ_SPLIT": ("Divisor", "Jogue uma mao dividida"),
416     "BJ_DEALER_BUST": ("Banca Estourou", "Force a banca a estourar"),
417     "ROUL_STRAIGHT": ("Numero da Sorte", "Ganhe uma aposta em numero"),
418     "ROUL_ZERO": ("Le Zero", "Acerte o zero apostado"),
419     "SLOT_FREE_SPINS": ("Rodada Bonus", "Ative giros gratis"),
420     "SLOT_FIVE_KIND": ("Rolos Cheios", "Consiga 5 iguais"),
421     "PREM_MINOR": ("Jackpot Menor", "Ganhe o jackpot 3-8"),
422     "PREM_MAJOR": ("Jackpot Maior", "Ganhe o jackpot 3-9"),
423     "PREM_MAXIMUM": ("Jackpot Maximo", "Ganhe o jackpot 8-9"),
424     "POKER_FULL": ("Full House", "Consiga Full House"),
425     "POKER_FOUR": ("Quadra", "Consiga uma Quadra"),
426     "POKER_STRAIGHT_FLUSH": ("Straight Flush", "Consiga um Straight Flush"),
427     "POKER_ROYAL": ("Coroa Real", "Consiga um Royal Flush"),
428 },
429 "RO": {
430     "FIRST_WIN": ("Prima Victorie", "Castiga prima runda"),
431     "BIG_WIN": ("Castig Mare", "Castiga 500+ intr-o runda"),
432     "HUGE_WIN": ("Castig Urias", "Castiga 2000+ intr-o runda"),
433     "MILLIONAIRE": ("Milionar", "Ajunge la $10.000"),
434     "BANKRUPT": ("La Pamant", "Ajungi la $0 si resetare"),
435     "VETERAN": ("Veteran", "Joaca 100 de runde"),
436     "HIGH_ROLLER": ("Jucator Mare", "Pariaza $250 dintr-o data"),
437     "BJ_NATURAL": ("21 Natural", "Obtine un Blackjack (A+10)"),
438     "BJ_DOUBLE_WIN": ("Dublare", "Castiga un Double Down"),
439     "BJ_SPLIT": ("Impartitor", "Joaca o mana impartita"),
440     "BJ_DEALER_BUST": ("Banca Cade", "Fa dealerul sa depaseasca"),
441     "ROUL_STRAIGHT": ("Numar Norocos", "Castiga un pariu pe numar"),
442     "ROUL_ZERO": ("Le Zero", "Nimereste zero cu pariu"),
443     "SLOT_FREE_SPINS": ("Runda Bonus", "Declanseaza rotiri gratis"),
444     "SLOT_FIVE_KIND": ("Role Pline", "Potriveste 5 la fel"),
445     "PREM_MINOR": ("Jackpot Minor", "Ia jackpotul 3-8"),
446     "PREM_MAJOR": ("Jackpot Major", "Ia jackpotul 3-9"),
447     "PREM_MAXIMUM": ("Jackpot Maxim", "Ia jackpotul 8-9"),
448     "POKER_FULL": ("Full House", "Obtine Full House"),
```

PY16CASINO - CODE LISTING (PAGE 8)

```
449         "POKER_FOUR": ("Careu", "Obtine un Careu"),
450         "POKER_STRAIGHT_FLUSH": ("Chinta de Culoare", "Obtine o Chinta de Culoare"),
451         "POKER_ROYAL": ("Coroana Regala", "Obtine o Chinta Regala"),
452     },
453 }
454
455 def tr(key):
456     """UI-String in der aktuellen Sprache. Faellt auf EN, dann auf den Key selbst zurueck.
457     cur = TR.get(language)
458     if cur and key in cur:
459         return cur[key]
460     return TR["EN"].get(key, key)
461
462 def tr_ach(ach_id):
463     """(name, desc) eines Achievements in der aktuellen Sprache. EN-Fallback aus ACHIEVEME
464     if language != "EN":
465         loc = ACH_TR.get(language)
466         if loc and ach_id in loc:
467             return loc[ach_id]
468     for a in ACHIEVEMENTS:
469         if a[0] == ach_id:
470             return (a[1], a[2])
471     return (ach_id, "")
472
473 # Beim Modul-Laden sofort versuchen, alte Stats zu laden
474 _save_existed = load_state()
475
476 # Beim echten Erststart (kein Save vorhanden) zeigen wir zuerst die
477 # Schwierigkeitswahl. Sonst direkt ins Hauptmenue mit geladener Bankroll.
478 if not _save_existed:
479     state = "DIFFICULTY_SELECT"
480
481 # Betting Setup
482 bets = [10, 50, 100, 250]
483 chip_colors = [12, 8, 11, 2] # Blau, Rot, Gruen, Lila
484 bet_idx = 0
485 current_bet = 0
486 current_bet_color = 0
487
488 # Action Menu Setup
489 action_idx = 0
490
491 # Video Poker State
492 poker_held = [False, False, False, False, False]
493 poker_cursor = 0
494
495 # Roulette State
496 roulette_cursor_x = 1
497 roulette_cursor_y = 2
498 roulette_bets = {}
499 roulette_result = 0
500 roulette_angle = 0.0
501 roulette_speed = 0.0
502 wheel_angle = 0.0 # Winkel des inneren Kessels
503 roulette_lower_pref = 0 # 0 = linke Spalte des Paares (1-18/RED/ODD), 1 = rechte (EVEN/BLA
504 RED_NUMS = [1,3,5,7,9,12,14,16,18,19,21,23,25,27,30,32,34,36]
505 roulette_sequence = [0, 32, 15, 19, 4, 21, 2, 25, 17, 34, 6, 27, 13, 36, 11, 30, 8, 23, 10
506
507 # --- Slots State & Reel Strips ---
508 slot_symbols = ["KIR", "ZIT", "ORG", "PFL", "GLO", "DIA", " 7 ", "BNS"]
509 slot_colors = {"KIR": 8, "ZIT": 10, "ORG": 9, "PFL": 2, "GLO": 10, "DIA": 7, " 7 ": 8, "BN
510 slot_payouts = {
511     "KIR": {3: 2, 4: 5, 5: 10}, "ZIT": {3: 2, 4: 5, 5: 15}, "ORG": {3: 3, 4: 8, 5: 20},
512     "PFL": {3: 3, 4: 10, 5: 25}, "GLO": {3: 5, 4: 15, 5: 50}, "DIA": {3: 10, 4: 30, 5: 100
```

PY16CASINO - CODE LISTING (PAGE 9)

```
513     " 7 ": {3: 15, 4: 50, 5: 250}
514 }
515 slot_lines = [
516     [(1,0), (1,1), (1,2), (1,3), (1,4)], [(0,0), (0,1), (0,2), (0,3), (0,4)],
517     [(2,0), (2,1), (2,2), (2,3), (2,4)], [(0,0), (1,1), (2,2), (1,3), (0,4)],
518     [(2,0), (1,1), (0,2), (1,3), (2,4)], [(0,0), (0,1), (1,2), (0,3), (0,4)],
519     [(2,0), (2,1), (1,2), (2,3), (2,4)]
520 ]
521
522 slot_strips = [
523     ["ZIT", "KIR", "BNS", "KIR", "ZIT", "DIA", "PFL", "GLO", "ORG", "PFL", "GLO", "ORG", "
524     ["KIR", "ORG", "PFL", "ORG", "KIR", "ORG", "DIA", "ORG", "ZIT", "ZIT", "PFL", "KIR", "
525     ["PFL", "GLO", "ZIT", "ORG", "KIR", "PFL", "KIR", "GLO", "PFL", "KIR", "KIR", "ZIT", "
526     ["PFL", "GLO", "ORG", "ORG", " 7 ", "ORG", "PFL", "KIR", "KIR", "GLO", "PFL", "DIA", "
527     ["PFL", "PFL", "ZIT", "ZIT", "KIR", "DIA", "KIR", "DIA", "GLO", "PFL", "BNS", "PFL", "
528 ]
529
530 # --- Custom Logo Design fuer das Hauptmenue ---
531 logo_py16 = [
532     "PPP Y   Y --- 1 666 ",
533     "P  P Y Y      11 6  ",
534     "PPP  Y   --- 1 666 ",
535     "P    Y      1 6 6",
536     "P    Y   --- 111 666 "
537 ]
538 logo_casino = [
539     "CCC  A  SSS III N   N 000",
540     "C    A A S   I  NN N 0 0",
541     "C    AAA SSS I  N N N 0 0",
542     "C    A A  S I  N  NN 0 0",
543     "CCC A A SSS III N   N 000"
544 ]
545
546 slots_grid = [["KIR"] * 5 for _ in range(3)]
547 slot_current_idx = [0, 0, 0, 0, 0]
548 slot_pixel_offset = [0.0, 0.0, 0.0, 0.0, 0.0]
549 slot_target_stops = [0, 0, 0, 0, 0]
550 active_coins = []
551 slot_winning_lines = []
552 slot_winning_scatters = []
553 slot_free_spins = 0
554
555 deck = []
556 player_hand = []
557 dealer_hand = []
558 msg = ""
559 msg2 = ""
560 timer = 0
561
562 # --- Blackjack: Mehrere Haende (fuer Split) und per-Hand-Einsaetze (fuer Double) ---
563 bj_hands = [] # Liste von Haenden; jede Hand ist eine Liste von Karten
564 bj_bets = [] # Einsatz pro Hand (kann nach DOUBLE verdoppelt sein)
565 bj_hand_idx = 0 # aktiver Hand-Index waehrend PLAYER_TURN
566
567 # Sterne-Cache fuer Menue-Hintergrund (einmalig vorberechnet,
568 # damit der globale random.seed nicht jeden Frame manipuliert wird)
569 _star_rng = random.Random(12345)
570 star_cache = []
571 for _ in range(50):
572     sx = _star_rng.randint(0, 255)
573     sy = _star_rng.randint(0, 110)
574     sc = _star_rng.choice([5, 6, 7])
575     twink = _star_rng.random() < 0.05
576     star_cache.append((sx, sy, sc, twink))
```

PY16CASINO - CODE LISTING (PAGE 10)

```
577
578 # =====
579 # HINTERGRUNDMUSIK (interner Tracker, laeuft ueber py16.tone)
580 # =====
581 # Pro Zeile zwei Spuren: (bass_midi, lead_midi). None = kein neuer Anschlag.
582 # Bass laeuft auf Dreieckswelle (Kanal 6), Lead auf Square mit PWM (Kanal 7).
583 # Bei 10 Frames/Zeile -> ca. 90 BPM (16-tel-Aufloesung).
584
585 MUSIC_FRAMES_PER_ROW = 10
586 music_row = 0
587 music_frame_in_row = 0
588 _last_music_kind = None
589
590 def _midi_hz(n):
591     if n is None: return 0
592     return 440.0 * (2 ** ((n - 69) / 12.0))
593
594 # Menue-Musik: jazziges Cm7 - Fm7 - Bb7 - EbMaj7 (Casino-Lounge)
595 MUSIC_MENU = [
596     # Cm7
597     (48, 75), (None, None), (None, 70), (None, None),
598     (43, 79), (None, None), (None, 75), (None, None),
599     # Fm7
600     (53, 80), (None, None), (None, 75), (None, None),
601     (48, 84), (None, None), (None, 80), (None, None),
602     # Bb7
603     (46, 77), (None, None), (None, 74), (None, None),
604     (41, 82), (None, None), (None, 77), (None, None),
605     # EbMaj7
606     (51, 79), (None, None), (None, 75), (None, None),
607     (46, 82), (None, None), (None, 79), (None, None),
608 ]
609
610 # Tisch-Musik: nur sparsamer Bass-Puls, soll nicht ablenken
611 MUSIC_TABLE = [
612     (36, None), (None, None), (None, None), (None, None),
613     (None, None), (None, None), (None, None), (None, None),
614     (41, None), (None, None), (None, None), (None, None),
615     (None, None), (None, None), (None, None), (None, None),
616     (34, None), (None, None), (None, None), (None, None),
617     (None, None), (None, None), (None, None), (None, None),
618     (39, None), (None, None), (None, None), (None, None),
619     (None, None), (None, None), (None, None), (None, None),
620 ]
621
622 def tick_music():
623     """Pro Frame einmal aufrufen. Triggert Noten je nach aktuellem state."""
624     global music_row, music_frame_in_row, _last_music_kind
625     if not sound_enabled:
626         # Bei abgeschaltetem Sound die Position einfrieren
627         return
628
629     # Aktuellen Song anhand des Game-States waehlen
630     if state == "MENU":
631         track, kind = MUSIC_MENU, "menu"
632     elif state in ["SLOTS_PAYTABLE", "POKER_PAYTABLE"]:
633         track, kind = None, "none"
634     else:
635         track, kind = MUSIC_TABLE, "table"
636
637     # Bei Wechsel des Songs sauber auf Zeile 0 setzen
638     if kind != _last_music_kind:
639         music_row = 0
640         music_frame_in_row = 0
```

PY16CASINO - CODE LISTING (PAGE 11)

```
641         _last_music_kind = kind
642
643     if track is None:
644         return
645
646     if music_frame_in_row == 0:
647         bass, lead = track[music_row % len(track)]
648         if bass is not None:
649             py16.tone(_midi_hz(bass), 700, py16.WAVE_TRIANGLE,
650                       attack_ms=30, decay_ms=300, sustain=0.45, release_ms=400,
651                       channel=6)
652         if lead is not None:
653             py16.tone(_midi_hz(lead), 350, py16.WAVE_SQUARE,
654                       attack_ms=20, decay_ms=80, sustain=0.30, release_ms=150,
655                       pulse_width=0.25, channel=7)
656
657     music_frame_in_row += 1
658     if music_frame_in_row >= MUSIC_FRAMES_PER_ROW:
659         music_frame_in_row = 0
660         music_row = (music_row + 1) % len(track)
661
662 # =====
663 # ACHIEVEMENT-TOAST (Update + Rendering)
664 # =====
665 def tick_achievements():
666     """Frame-Tick: laufenden Toast runterzaehlen, naechsten aus Queue starten."""
667     global achievement_toast_timer
668     if achievement_toast_timer > 0:
669         achievement_toast_timer -= 1
670     if achievement_toast_timer <= 0 and achievement_toast_queue:
671         # Naechsten aus Queue starten, Sound spielen
672         achievement_toast_timer = ACH_TOAST_DURATION
673         if sound_enabled:
674             # Kurze 3-Noten-Triumph-Fanfare auf Square-Welle
675             py16.tone(880, 80, py16.WAVE_SQUARE, decay_ms=80, channel=2)
676             py16.tone(1175, 80, py16.WAVE_SQUARE, decay_ms=80, channel=3, pulse_width=0.3)
677             py16.tone(1760, 200, py16.WAVE_SQUARE, decay_ms=200, channel=4)
678
679 def draw_achievement_toast():
680     """Toast unten am Bildschirm. Aus Queue[0], solange achievement_toast_timer > 0."""
681     if achievement_toast_timer <= 0 or not achievement_toast_queue:
682         return
683     a = achievement_toast_queue[0]
684     _id = a[0]
685     name, _desc = tr_ach(_id) # uebersetzter Achievement-Name
686     # Slide-in/out per Y-Offset
687     remaining = achievement_toast_timer
688     if remaining > ACH_TOAST_DURATION - ACH_TOAST_FADE:
689         # Slide-in
690         progress = (ACH_TOAST_DURATION - remaining) / ACH_TOAST_FADE
691         y_off = int((1 - progress) * 30)
692     elif remaining < ACH_TOAST_FADE:
693         # Slide-out
694         progress = remaining / ACH_TOAST_FADE
695         y_off = int((1 - progress) * 30)
696     else:
697         y_off = 0
698
699     bx, by = 40, 178 + y_off
700     bw, bh = 176, 30
701     # Goldener Rahmen + dunkler Hintergrund
702     py16.rectfill(bx, by, bw, bh, 0)
703     py16.rect(bx, by, bw, bh, 10)
704     py16.rect(bx + 1, by + 1, bw - 2, bh - 2, 10)
```

PY16CASINO - CODE LISTING (PAGE 12)

```
705 # Stern-Icon links (einfaches Pixel-Cluster)
706 sx, sy = bx + 8, by + bh // 2 - 4
707 py16.pset(sx + 3, sy, 10)
708 py16.pset(sx + 3, sy + 1, 10); py16.pset(sx + 2, sy + 2, 10); py16.pset(sx + 4, sy + 2
709 py16.rectfill(sx, sy + 3, 7, 1, 10)
710 py16.pset(sx + 1, sy + 4, 10); py16.pset(sx + 5, sy + 4, 10)
711 py16.pset(sx + 2, sy + 5, 10); py16.pset(sx + 4, sy + 5, 10)
712 py16.pset(sx + 1, sy + 6, 10); py16.pset(sx + 5, sy + 6, 10)
713 # Texte
714 py16.text(tr("toast_unlocked"), bx + 22, by + 5, 10)
715 py16.text(name, bx + 22, by + 17, 7)
716
717 # Wenn Toast abgelaufen, Queue weiterruecken
718 if achievement_toast_timer <= 1:
719     if achievement_toast_queue:
720         achievement_toast_queue.pop(0)
721
722 def play_sound(sfx_name):
723     global sound_enabled
724     if not sound_enabled: return
725     if sfx_name == "blip": py16.tone(600, 20, py16.WAVE_TRIANGLE)
726     elif sfx_name == "chip": py16.tone(800, 40, py16.WAVE_SQUARE, decay_ms=40)
727     elif sfx_name == "card": py16.tone(1000, 60, py16.WAVE_NOISE, attack_ms=10, decay_ms=5
728     elif sfx_name == "error": py16.tone(150, 150, py16.WAVE_SAW, decay_ms=100)
729     elif sfx_name == "win": py16.tone(660, 200, py16.WAVE_SQUARE, attack_ms=10, decay_ms=1
730     elif sfx_name == "jackpot": py16.tone(880, 400, py16.WAVE_SQUARE, pulse_width=0.125, a
731     elif sfx_name == "lose": py16.tone(200, 300, py16.WAVE_SAW, decay_ms=250)
732     elif sfx_name == "coin": py16.tone(1200, 20, py16.WAVE_SQUARE, decay_ms=20, channel=1)
733
734 def init_deck():
735     global deck
736     deck = []
737     if game_mode == "PREMIUM":
738         for v in range(1, 11):
739             for s in range(4): deck.append((v, s))
740     elif game_mode in ["POKER", "BLACKJACK"]:
741         for v in range(2, 15):
742             for s in range(4): deck.append((v, s))
743     random.shuffle(deck)
744
745 def draw_card_from_deck():
746     global deck
747     if len(deck) == 0: init_deck()
748     return deck.pop()
749
750 def end_game_reset():
751     global game_started, state, msg, msg2, active_coins, slot_winning_lines, slot_winning_
752     global bj_hands, bj_bets, bj_hand_idx, _money_at_round_start
753     money += sum(roulette_bets.values())
754     roulette_bets.clear()
755     game_started = False
756     state = "MENU"
757     msg, msg2 = "", ""
758     slot_free_spins = 0
759     active_coins, slot_winning_lines, slot_winning_scatters = [], [], []
760     bj_hands, bj_bets, bj_hand_idx = [], [], 0
761     _money_at_round_start = None # Abbruch zaehlt nicht als gespielte Runde
762     save_state()
763
764 # --- PREMIUM JACKPOT LOGIK ---
765 def calc_score(hand): return sum(card[0] for card in hand) % 10
766 def check_premium(hand):
767     vals = sorted([c[0] for c in hand])
768     if len(vals) == 3 and vals == [3, 8, 9]:
```

PY16CASINO - CODE LISTING (PAGE 13)

```
769         first_two = sorted([hand[0][0], hand[1][0]])
770         if first_two == [3, 8]: return "MINOR", 0.20
771         if first_two == [3, 9]: return "MAJOR", 0.50
772         if first_two == [8, 9]: return "MAXIMUM", 0.95
773     return None, 0
774
775 def evaluate_premium():
776     global state, money, jackpot, msg, msg2
777     pscore = calc_score(player_hand)
778     dscore = calc_score(dealer_hand)
779     premium_type, premium_pct = check_premium(player_hand)
780     vals = sorted([c[0] for c in player_hand])
781
782     is_kuppin = len(player_hand) == 2 and vals == [1, 9]
783     is_shippin = len(player_hand) == 3 and vals == [4, 4, 4]
784     is_shigoro = len(player_hand) == 3 and vals == [4, 5, 6]
785
786     if premium_type:
787         win_amount = int(jackpot * premium_pct)
788         jackpot -= win_amount; money += win_amount
789         msg = "PREMIUM " + premium_type + "!"; msg2 = "WON " + str(win_amount) + "!"
790         play_sound("jackpot")
791         if premium_type == "MINOR": unlock("PREM_MINOR")
792         elif premium_type == "MAJOR": unlock("PREM_MAJOR")
793         elif premium_type == "MAXIMUM": unlock("PREM_MAXIMUM")
794     elif is_shippin:
795         money += current_bet * 5; msg = "SHIPPIN! (4-4-4)"; msg2 = "WON " + str(current_bet)
796         play_sound("jackpot")
797     elif is_shigoro:
798         money += current_bet * 3; msg = "SHIGORO! (4-5-6)"; msg2 = "WON " + str(current_bet)
799         play_sound("win")
800     elif is_kuppin:
801         money += current_bet * 2; msg = "KUPPIN! (1-9)"; msg2 = "WON " + str(current_bet)
802         play_sound("win")
803     elif pscore > dscore:
804         money += current_bet; msg = "YOU WIN!"; msg2 = "SCORE: " + str(pscore) + " VS " +
805         play_sound("win")
806     else:
807         jackpot += int(current_bet * 0.20); money -= current_bet
808         if pscore == dscore: msg = "TIE GOES TO DEALER."
809         else: msg = "DEALER WINS."
810         msg2 = "SCORE: " + str(pscore) + " VS " + str(dscore)
811         play_sound("lose")
812     state = "GAME_OVER"
813
814 # --- BLACKJACK LOGIK ---
815 def calc_bj_score(hand):
816     score, aces = 0, 0
817     for card in hand:
818         val = card[0]
819         if 11 <= val <= 13: score += 10
820         elif val == 14: aces += 1; score += 11
821         else: score += val
822     while score > 21 and aces > 0:
823         score -= 10; aces -= 1
824     return score
825
826 def bj_card_value(card):
827     """Blackjack-Wert einer einzelnen Karte (Bilder = 10, Ass = 11). Fuer Split-Pruefung."
828     val = card[0]
829     if 11 <= val <= 13: return 10
830     if val == 14: return 11
831     return val
832
```

PY16CASINO - CODE LISTING (PAGE 14)

```
833 def bj_total_at_risk():
834     """Summe der aktuell ueber alle BJ-Haende eingesetzten Chips."""
835     return sum(bj_bets) if bj_bets else 0
836
837 def bj_can_commit(amount):
838     """Reicht das Guthaben, um zusaetzlich 'amount' an Einsatz zu binden?"""
839     return money >= bj_total_at_risk() + amount
840
841 def bj_available_actions():
842     """Aktuell waehlbare Aktionen fuer die aktive BJ-Hand."""
843     actions = ["HIT", "STAND"]
844     if game_mode != "BLACKJACK" or not bj_hands:
845         return actions
846     cur_hand = bj_hands[bj_hand_idx]
847     if len(cur_hand) == 2:
848         cur_bet = bj_bets[bj_hand_idx]
849         # DOUBLE: aktuellen Einsatz nochmal binden
850         if bj_can_commit(cur_bet):
851             actions.append("DOUBLE")
852         # SPLIT: nur einmal pro Runde, gleicher Kartenwert, ein Grundeinsatz mehr noetig
853         if (bj_card_value(cur_hand[0]) == bj_card_value(cur_hand[1])
854             and len(bj_hands) < 2
855             and bj_can_commit(current_bet)):
856             actions.append("SPLIT")
857     return actions
858
859 def advance_bj_hand():
860     """Naechste BJ-Hand aktivieren oder zum Dealer wechseln. Springt auto bei 21+."""
861     global bj_hand_idx, state, timer, player_hand, action_idx
862     if bj_hand_idx + 1 < len(bj_hands):
863         bj_hand_idx += 1
864         player_hand = bj_hands[bj_hand_idx]
865         action_idx = 0
866         # Wenn die neue Hand bereits 21 oder mehr hat, sofort weiterspringen
867         if calc_bj_score(player_hand) >= 21:
868             advance_bj_hand()
869     else:
870         state = "DEALER_TURN"
871         timer = 30
872
873 def evaluate_blackjack():
874     global state, money, jackpot, msg, msg2
875     dscore = calc_bj_score(dealer_hand)
876     d_bj = (dscore == 21 and len(dealer_hand) == 2)
877     is_split = len(bj_hands) > 1
878
879     total_change = 0
880     natural_bj = False
881     sub_msgs = []
882
883     for h_idx, hand in enumerate(bj_hands):
884         bet = bj_bets[h_idx]
885         p_score = calc_bj_score(hand)
886         # Natural-Blackjack zaehlt nur ohne Split (Casino-Standard)
887         p_bj = (p_score == 21 and len(hand) == 2) and not is_split
888         label = "H{}".format(h_idx + 1)
889
890         # Achievement-Trigger: Double-Win, wenn der Hand-Einsatz > Grundeinsatz und sie ni
891         doubled = (bet > current_bet)
892
893         if p_score > 21:
894             total_change -= bet
895             jackpot += int(bet * 0.10)
896             sub_msgs.append(label + " BUST")
```

PY16CASINO - CODE LISTING (PAGE 15)

```
897         elif dscore > 21:
898             total_change += bet
899             sub_msgs.append(label + " WIN")
900             unlock("BJ_DEALER_BUST")
901             if doubled: unlock("BJ_DOUBLE_WIN")
902         elif p_bj and not d_bj:
903             win_amt = int(bet * 1.5)
904             total_change += win_amt
905             natural_bj = True
906             sub_msgs.append(label + " BJ")
907             unlock("BJ_NATURAL")
908         elif p_score > d_score:
909             total_change += bet
910             sub_msgs.append(label + " WIN")
911             if doubled: unlock("BJ_DOUBLE_WIN")
912         elif d_score > p_score:
913             total_change -= bet
914             jackpot += int(bet * 0.10)
915             sub_msgs.append(label + " LOSE")
916         else:
917             sub_msgs.append(label + " PUSH")
918
919     money += total_change
920
921     if is_split:
922         # Beide Hand-Ausgaenge anzeigen
923         msg = " ".join(sub_msgs)
924         if total_change > 0: msg2 = "NET +" + str(total_change)
925         elif total_change < 0: msg2 = "NET -" + str(-total_change)
926         else: msg2 = "PUSH"
927     else:
928         # Einzelne Hand: Originalstil bewahren
929         s = sub_msgs[0]
930         if "BUST" in s:
931             msg = "BUST!"; msg2 = "DEALER WINS."
932         elif "BJ" in s:
933             msg = "BLACKJACK!"; msg2 = "WON " + str(total_change) + "!"
934         elif "WIN" in s:
935             # Unterscheidung: Dealer-Bust vs. einfacher Win
936             if d_score > 21: msg = "DEALER BUSTS!"
937             else: msg = "YOU WIN!"
938             msg2 = "WON " + str(total_change) + "!"
939         elif "LOSE" in s:
940             msg = "DEALER WINS."; msg2 = "LOST " + str(-total_change)
941         else:
942             msg = "PUSH (TIE)"; msg2 = "BET RETURNED."
943
944     # Sound
945     if natural_bj: play_sound("jackpot")
946     elif total_change > 0: play_sound("win")
947     elif total_change < 0: play_sound("lose")
948     else: play_sound("blip")
949
950     state = "GAME_OVER"
951
952     # --- VIDEO POKER LOGIK ---
953     def evaluate_poker():
954         global state, money, jackpot, msg, msg2, active_coins
955         vals = sorted([c[0] for c in player_hand])
956         suits = [c[1] for c in player_hand]
957         is_flush = len(set(suits)) == 1
958         is_straight = vals == list(range(vals[0], vals[0]+5))
959         if vals == [2, 3, 4, 5, 14]: is_straight = True
960
```

PY16CASINO - CODE LISTING (PAGE 16)

```
961     counts = {v: vals.count(v) for v in set(vals)}
962     freqs = sorted(list(counts.values()), reverse=True)
963     is_royal = is_flush and vals == [10, 11, 12, 13, 14]
964     pairs = [v for v, count in counts.items() if count == 2]
965     jacks_or_better = any(v >= 11 for v in pairs)
966
967     payout, payout_msg = 0, ""
968     if is_royal:
969         payout = jackpot; jackpot = 5000; payout_msg = "ROYAL FLUSH!"
970         unlock("POKER_ROYAL")
971     elif is_straight and is_flush:
972         payout = current_bet * 50; payout_msg = "STRAIGHT FLUSH!"
973         unlock("POKER_STRAIGHT_FLUSH")
974     elif freqs == [4, 1]:
975         payout = current_bet * 25; payout_msg = "4 OF A KIND!"
976         unlock("POKER_FOUR")
977     elif freqs == [3, 2]:
978         payout = current_bet * 9; payout_msg = "FULL HOUSE!"
979         unlock("POKER_FULL")
980     elif is_flush: payout = current_bet * 6; payout_msg = "FLUSH!"
981     elif is_straight: payout = current_bet * 4; payout_msg = "STRAIGHT!"
982     elif freqs == [3, 1, 1]: payout = current_bet * 3; payout_msg = "3 OF A KIND!"
983     elif freqs == [2, 2, 1]: payout = current_bet * 2; payout_msg = "TWO PAIR!"
984     elif jacks_or_better and freqs == [2, 1, 1, 1]: payout = current_bet * 1; payout_msg =
985
986     if payout > 0:
987         money += payout; msg = payout_msg; msg2 = "WON " + str(payout) + "!"
988         if payout >= current_bet * 25: play_sound("jackpot")
989         else: play_sound("win")
990
991         num_coins = min(100, int(payout / max(1, current_bet) * 5))
992         if num_coins < 5: num_coins = 5
993         for _ in range(num_coins):
994             active_coins.append([128 + random.uniform(-20, 20), 156 + random.uniform(0, 4)
995
996     else:
997         money -= current_bet; msg = "NO WIN"; msg2 = "LOST " + str(current_bet); play_soun
998         state = "GAME_OVER"
999
1000 # --- ROULETTE LOGIK ---
1001 def get_spot_from_cursor(cx, cy):
1002     if cy < 3:
1003         if cx == 0: return "0"
1004         if cx == 13: return f"COL{3-cy}"
1005         return str((cx-1)*3 + (3-cy))
1006     if cy == 3: return ["1ST12", "2ND12", "3RD12"][cx-1]
1007     if cy == 4: return ["1-18", "EVEN", "RED", "BLACK", "ODD", "19-36"][cx-1]
1008     return None
1009
1010 def get_target_rect(target):
1011     if target == "0": return (23, 22, 15, 48)
1012     for cy in range(3):
1013         for cx in range(1, 14):
1014             spot = str((cx-1)*3 + (3-cy)) if cx <= 12 else f"COL{3-cy}"
1015             if spot == target: return (38 + (cx-1)*15, 22 + cy*16, 15, 16)
1016     for cx in range(1, 4):
1017         if ["1ST12", "2ND12", "3RD12"][cx-1] == target: return (38 + (cx-1)*60, 70, 60, 16)
1018     for cx in range(1, 7):
1019         if ["1-18", "EVEN", "RED", "BLACK", "ODD", "19-36"][cx-1] == target: return (38 +
1020
1021     return (0, 0, 0, 0)
1022
1023 def get_roulette_color(t):
1024     if t == "0": return 11
1025     if t.isdigit() and int(t) in RED_NUMS: return 8
1026     if t.isdigit() and int(t) not in RED_NUMS: return 5
```

PY16CASINO - CODE LISTING (PAGE 17)

```
1025     if t == "RED": return 8
1026     if t == "BLACK": return 5
1027     return 1
1028
1029 def evaluate_roulette():
1030     global state, money, jackpot, msg, msg2, roulette_history
1031     payout = 0
1032     res = roulette_result
1033
1034     # Ergebnis in die Historie eintragen (neueste vorn, Buffer auf Max-Laenge halten)
1035     roulette_history.insert(0, int(res))
1036     if len(roulette_history) > ROULETTE_HISTORY_MAX:
1037         roulette_history = roulette_history[:ROULETTE_HISTORY_MAX]
1038
1039     for target, amount in roulette_bets.items():
1040         win = False
1041         mult = 0
1042         if str(target) == str(res):
1043             win = True; mult = 36
1044             # Achievement: Straight-Up-Treffer auf irgendeine Zahl
1045             unlock("ROUL_STRAIGHT")
1046             # Sonder-Achievement: gerade auf die Null gesetzt und Null kam
1047             if str(target) == "0":
1048                 unlock("ROUL_ZERO")
1049             elif target == "COL3" and res % 3 == 0 and res != 0: win = True; mult = 3
1050             elif target == "COL2" and res % 3 == 2 and res != 0: win = True; mult = 3
1051             elif target == "COL1" and res % 3 == 1 and res != 0: win = True; mult = 3
1052             elif target == "1ST12" and 1 <= res <= 12: win = True; mult = 3
1053             elif target == "2ND12" and 13 <= res <= 24: win = True; mult = 3
1054             elif target == "3RD12" and 25 <= res <= 36: win = True; mult = 3
1055             elif target == "1-18" and 1 <= res <= 18: win = True; mult = 2
1056             elif target == "19-36" and 19 <= res <= 36: win = True; mult = 2
1057             elif target == "RED" and res in RED_NUMS: win = True; mult = 2
1058             elif target == "BLACK" and res != 0 and res not in RED_NUMS: win = True; mult = 2
1059             elif target == "EVEN" and res != 0 and res % 2 == 0: win = True; mult = 2
1060             elif target == "ODD" and res != 0 and res % 2 != 0: win = True; mult = 2
1061
1062             if win: payout += amount * mult
1063             else: jackpot += int(amount * 0.10)
1064
1065     if payout > 0:
1066         money += payout; msg = "WINNER: " + str(res); msg2 = "WON " + str(payout) + "!"
1067         if payout >= sum(roulette_bets.values()) * 5: play_sound("jackpot")
1068         else: play_sound("win")
1069     else:
1070         msg = "RESULT: " + str(res); msg2 = "NO WIN"; play_sound("lose")
1071     state = "GAME_OVER"
1072
1073 # --- SLOT MACHINE LOGIK ---
1074 def evaluate_slots():
1075     global state, money, jackpot, msg, msg2, active_coins
1076     global slot_winning_lines, slot_winning_scatters, slot_free_spins
1077
1078     if slot_free_spins > 0: slot_free_spins -= 1
1079     else: money -= current_bet; jackpot += int(current_bet * 0.10)
1080
1081     total_win = 0
1082     line_bet = max(1.0, current_bet / 10.0)
1083
1084     for line in slot_lines:
1085         sym = slots_grid[line[0][0]][line[0][1]]; count = 1
1086         for i in range(1, 5):
1087             if slots_grid[line[i][0]][line[i][1]] == sym: count += 1
1088             else: break
```

PY16CASINO - CODE LISTING (PAGE 18)

```
1089         if count >= 3 and sym != "BNS":
1090             total_win += int(slot_payouts[sym][count] * line_bet)
1091             slot_winning_lines.append(line)
1092             if count == 5: unlock("SLOT_FIVE_KIND")
1093
1094         b_cells = []
1095         for r in range(3):
1096             for c in range(5):
1097                 if slots_grid[r][c] == "BNS": b_cells.append((r, c))
1098
1099         triggered_free_spins = False
1100         if len(b_cells) >= 3:
1101             slot_free_spins += 10
1102             slot_winning_scatters = b_cells
1103             triggered_free_spins = True
1104             unlock("SLOT_FREE_SPINS")
1105
1106         if total_win > 0 or triggered_free_spins:
1107             if total_win > 0:
1108                 money += total_win
1109                 msg = "SLOT WINNER!"
1110                 msg2 = "WON " + str(total_win) + " COINS"
1111                 if triggered_free_spins: msg = "10 FREE SPINS!"
1112             else:
1113                 msg = "BONUS TRIGGERED!"
1114                 msg2 = "10 FREE SPINS WON!"
1115
1116             if total_win >= current_bet * 4 or triggered_free_spins: play_sound("jackpot")
1117             else: play_sound("win")
1118
1119             num_coins = min(100, int(total_win / line_bet * 2)) if total_win > 0 else 0
1120             if triggered_free_spins and num_coins < 20: num_coins = 20
1121             if total_win > 0 and num_coins < 5: num_coins = 5
1122             for _ in range(num_coins):
1123                 active_coins.append([128 + random.uniform(-20, 20), 156 + random.uniform(0, 4)])
1124         else:
1125             msg = "NO WIN"
1126             msg2 = "FREE SPIN" if slot_free_spins > 0 else "BET LOST"
1127             play_sound("lose")
1128         state = "GAME_OVER"
1129
1130     def init():
1131         for c in range(5):
1132             idx = random.randint(0, len(slot_strips[c])-1)
1133             slot_current_idx[c] = idx
1134             slots_grid[0][c] = slot_strips[c][idx]
1135             slots_grid[1][c] = slot_strips[c][(idx+1) % len(slot_strips[c])]
1136             slots_grid[2][c] = slot_strips[c][(idx+2) % len(slot_strips[c])]
1137
1138     def update():
1139         global state, prev_state, game_started, sound_enabled, menu_idx, game_mode
1140         global bet_idx, current_bet, current_bet_color, player_hand, dealer_hand
1141         global msg, msg2, timer, money, action_idx
1142         global poker_held, poker_cursor
1143         global roulette_cursor_x, roulette_cursor_y, roulette_result, roulette_angle, roulette
1144         global roulette_lower_pref
1145         global active_coins, slot_winning_lines, slot_winning_scatters, slot_current_idx, slot
1146         global roulette_bets, wheel_angle, slot_free_spins
1147         global bj_hands, bj_bets, bj_hand_idx
1148         global _money_at_round_start, lifetime_won, lifetime_lost, games_played, biggest_win
1149         global difficulty, diff_cursor, language
1150
1151         # Tag/Nacht Berechnung für das Menü
1152         hour = datetime.datetime.now().hour
```

PY16CASINO - CODE LISTING (PAGE 19)

```
1153     is_night = (hour >= 20 or hour < 8)
1154
1155     py16.particles_update()
1156     tick_music()
1157     tick_achievements()
1158
1159     # --- SCHWIERIGKEITSWAHL (nur beim allerersten Start) ---
1160     if state == "DIFFICULTY_SELECT":
1161         if py16.btnp('up'):
1162             diff_cursor = (diff_cursor - 1) % len(DIFFICULTIES); play_sound("blip")
1163         if py16.btnp('down'):
1164             diff_cursor = (diff_cursor + 1) % len(DIFFICULTIES); play_sound("blip")
1165         if py16.btnp('z') or py16.btnp('enter'):
1166             # Schwierigkeit uebernehmen, Start-Bankroll setzen, Save anlegen
1167             diff_id, _name, bankroll, _desc = DIFFICULTIES[diff_cursor]
1168             difficulty = diff_id
1169             money = bankroll
1170             play_sound("chip")
1171             save_state() # ab jetzt existiert ein Save -> Screen kommt nie wieder
1172             state = "MENU"
1173         return
1174
1175     if game_mode == "ROULETTE":
1176         wheel_angle -= 0.02
1177
1178     if len(active_coins) > 0:
1179         next_coins = []
1180         if py16.t() % 4 == 0: play_sound("coin")
1181         for c in active_coins:
1182             c[0] += c[2]; c[1] += c[3]; c[3] += 0.4
1183             if c[1] < 240: next_coins.append(c)
1184         active_coins = next_coins
1185
1186     # --- MENUE LOGIK ---
1187     if py16.btnp('enter') and state not in ["MENU", "SLOTS_PAYTABLE", "POKER_PAYTABLE"]:
1188         prev_state = state; state = "MENU"; menu_idx = 0; play_sound("blip"); return
1189
1190     if state == "MENU":
1191         # Feuerwerk spawnen nur bei Nacht
1192         if is_night and random.random() < 0.02:
1193             colors = [8, 9, 10, 11, 14]
1194             py16.burst(
1195                 random.randint(30, 226), random.randint(20, 80),
1196                 count=25, color=random.choice(colors), life=40,
1197                 speed=2.0, blend="add"
1198             )
1199
1200         if py16.btnp('enter') and game_started:
1201             state = prev_state; play_sound("blip"); return
1202         if py16.btnp('up'): menu_idx = max(0, menu_idx - 1); play_sound("blip")
1203         if py16.btnp('down'): menu_idx = min(4, menu_idx + 1); play_sound("blip")
1204
1205         if py16.btnp('left') or py16.btnp('right'):
1206             if menu_idx == 0 and not game_started:
1207                 shift = -1 if py16.btnp('left') else 1
1208                 game_mode = modes.index(game_mode) + shift % len(modes)
1209                 if game_mode not in ["SLOTS", "POKER"] and bet_idx > 3: bet_idx = 3
1210                 active_coins, slot_winning_lines, slot_winning_scatters = [], [], []
1211                 money += sum(roulette_bets.values()); roulette_bets.clear(); slot_free_spi
1212                 play_sound("blip")
1213             elif menu_idx == 2:
1214                 sound_enabled = not sound_enabled
1215                 if sound_enabled: play_sound("chip")
1216             elif menu_idx == 4:
```

PY16CASINO - CODE LISTING (PAGE 20)

```
1217         # Sprache durchschalten (L = rueckwaerts, R = vorwaerts)
1218         shift = -1 if py16.btnp('left') else 1
1219         cur = next((i for i, l in enumerate(LANGUAGES) if l[0] == language), 0)
1220         language = LANGUAGES[(cur + shift) % len(LANGUAGES)][0]
1221         play_sound("blip"); save_state()
1222
1223     if py16.btnp('z'):
1224         if menu_idx == 0 and not game_started:
1225             game_mode = modes[(modes.index(game_mode) + 1) % len(modes)]
1226             if game_mode not in ["SLOTS", "POKER"] and bet_idx > 3: bet_idx = 3
1227             active_coins, slot_winning_lines, slot_winning_scatters = [], [], []
1228             money += sum(roulette_bets.values()); roulette_bets.clear(); slot_free_spi
1229             play_sound("blip")
1230         elif menu_idx == 1:
1231             if not game_started:
1232                 play_sound("chip"); game_started = True; init_deck(); state = "BETTING
1233             else: play_sound("error"); end_game_reset()
1234         elif menu_idx == 2:
1235             sound_enabled = not sound_enabled
1236             if sound_enabled: play_sound("chip")
1237         elif menu_idx == 3:
1238             # Achievement-Liste oeffnen
1239             play_sound("blip"); state = "ACHIEVEMENTS"
1240         elif menu_idx == 4:
1241             # Sprache vorwaerts durchschalten
1242             cur = next((i for i, l in enumerate(LANGUAGES) if l[0] == language), 0)
1243             language = LANGUAGES[(cur + 1) % len(LANGUAGES)][0]
1244             play_sound("chip"); save_state()
1245         elif menu_idx == 0 and game_started:
1246             play_sound("chip"); state = prev_state
1247     return
1248
1249 # --- ACHIEVEMENT-LISTE (eigener Screen) ---
1250 if state == "ACHIEVEMENTS":
1251     global ach_cursor
1252     # Cursor: Auf/Ab um 1, Links/Rechts um Spalte (11) wechseln
1253     col_size = (ACH_TOTAL + 1) // 2 # 11 fuer 22 Achievements
1254     if py16.btnp('up'):
1255         ach_cursor = (ach_cursor - 1) % ACH_TOTAL; play_sound("blip")
1256     if py16.btnp('down'):
1257         ach_cursor = (ach_cursor + 1) % ACH_TOTAL; play_sound("blip")
1258     if py16.btnp('left'):
1259         if ach_cursor >= col_size:
1260             ach_cursor -= col_size; play_sound("blip")
1261     if py16.btnp('right'):
1262         if ach_cursor + col_size < ACH_TOTAL:
1263             ach_cursor += col_size; play_sound("blip")
1264     # Zurueck mit X, Space, Enter
1265     if py16.btnp('x') or py16.btnp('space') or py16.btnp('enter'):
1266         play_sound("blip"); state = "MENU"
1267     return
1268
1269 # --- WETT-PHASE ---
1270 if state == "BETTING":
1271     max_idx = 4 if game_mode in ["SLOTS", "POKER"] else 3
1272     if py16.btnp('left') or py16.btnp('up'):
1273         if bet_idx > 0: bet_idx -= 1; play_sound("blip")
1274     if py16.btnp('right') or py16.btnp('down'):
1275         if bet_idx < max_idx: bet_idx += 1; play_sound("blip")
1276
1277     if py16.btnp('z'):
1278         if game_mode == "SLOTS" and bet_idx == 4:
1279             play_sound("blip"); state = "SLOTS_PAYTABLE"; msg, msg2 = "", ""; return
1280         if game_mode == "POKER" and bet_idx == 4:
```

PY16CASINO - CODE LISTING (PAGE 21)

```
1281         play_sound("blip"); state = "POKER_PAYTABLE"; msg, msg2 = "", ""; return
1282
1283     current_bet = bets[bet_idx]
1284
1285     if game_mode == "ROULETTE":
1286         play_sound("blip"); current_bet_color = chip_colors[bet_idx]; state = "ROU
1287         roulette_cursor_x, roulette_cursor_y = 1, 2; msg, msg2 = "", ""; return
1288
1289     if money >= current_bet or (game_mode == "SLOTS" and slot_free_spins > 0):
1290         play_sound("chip"); current_bet_color = chip_colors[bet_idx]
1291         # Snapshot fuer Lifetime-Stats: Geldstand vor dieser Runde merken
1292         _money_at_round_start = money
1293         # High-Roller-Achievement: 250er-Chip gesetzt
1294         if current_bet >= 250: unlock("HIGH_ROLLER")
1295         if game_mode in ["PREMIUM", "BLACKJACK"]:
1296             init_deck()
1297             player_hand = [draw_card_from_deck(), draw_card_from_deck()]
1298             dealer_hand = [draw_card_from_deck(), draw_card_from_deck()]
1299             if game_mode == "BLACKJACK":
1300                 # Mehrhand-Struktur: zu Beginn genau eine Hand mit dem Grundeinsatz
1301                 bj_hands = [player_hand]
1302                 bj_bets = [current_bet]
1303                 bj_hand_idx = 0
1304                 state = "PLAYER_TURN"; action_idx = 0; py16.tone(1000, 60, py16.WAVE_N
1305                 if game_mode == "BLACKJACK" and calc_bj_score(player_hand) == 21: stat
1306             elif game_mode == "POKER":
1307                 init_deck(); player_hand = [draw_card_from_deck() for _ in range(5)];
1308                 state = "POKER_DRAW"; active_coins = []; py16.tone(1000, 60, py16.WAVE
1309             elif game_mode == "SLOTS":
1310                 state = "SLOTS_SPIN"; timer = 160
1311                 slot_target_stops = [random.randint(0, len(slot_strips[c])-1) for c in
1312                 active_coins, slot_winning_lines, slot_winning_scatters = [], [], []
1313                 msg, msg2 = "", ""
1314             else: play_sound("error")
1315
1316     elif state in ["SLOTS_PAYTABLE", "POKER_PAYTABLE"]:
1317         if py16.btnp('z') or py16.btnp('x') or py16.btnp('space'): play_sound("blip"); sta
1318
1319     elif state == "SLOTS_SPIN":
1320         timer -= 1
1321         for c in range(5):
1322             if timer < 160 - (c + 1) * 20:
1323                 slot_current_idx[c] = slot_target_stops[c]; slot_pixel_offset[c] = 0.0
1324             else:
1325                 slot_pixel_offset[c] += 8.0
1326                 if slot_pixel_offset[c] >= 34.0:
1327                     slot_pixel_offset[c] -= 34.0
1328                     slot_current_idx[c] = (slot_current_idx[c] - 1) % len(slot_strips[c])
1329                 idx = slot_current_idx[c]
1330                 slots_grid[0][c] = slot_strips[c][idx]
1331                 slots_grid[1][c] = slot_strips[c][(idx+1) % len(slot_strips[c])]
1332                 slots_grid[2][c] = slot_strips[c][(idx+2) % len(slot_strips[c])]
1333
1334         if timer % 5 == 0 and timer > 40: play_sound("card")
1335         if timer <= 0: evaluate_slots()
1336
1337     elif state == "ROULETTE_BOARD":
1338         cx, cy = roulette_cursor_x, roulette_cursor_y
1339         if py16.btnp('left'):
1340             if cy < 3: cx = max(0, cx - 1)
1341             elif cy == 3: cx = max(1, cx - 1)
1342             elif cy == 4: cx = max(1, cx - 1)
1343             play_sound("blip")
1344         if py16.btnp('right'):
```

PY16CASINO - CODE LISTING (PAGE 22)

```
1345         if cy < 3: cx = min(13, cx + 1)
1346         elif cy == 3: cx = min(3, cx + 1)
1347         elif cy == 4: cx = min(6, cx + 1)
1348         play_sound("blip")
1349     if py16.btnp('up'):
1350         if cy == 4:
1351             # Merken, ob wir auf der linken (1-18/RED/ODD) oder rechten (EVEN/BLACK/19
1352             roulette_lower_pref = (cx - 1) % 2
1353             cy = 3; cx = (cx-1)//2 + 1
1354         elif cy == 3: cy = 2; cx = (cx-1)*4 + 2
1355         elif cy > 0: cy -= 1
1356         play_sound("blip")
1357     if py16.btnp('down'):
1358         if cy < 2: cy += 1
1359         elif cy == 2:
1360             if cx == 0: cx = 1; cy = 3
1361             elif cx == 13: cx = 3; cy = 3
1362             else: cy = 3; cx = (cx-1)//4 + 1
1363         elif cy == 3: cy = 4; cx = (cx-1)*2 + 1 + roulette_lower_pref
1364         play_sound("blip")
1365
1366     roulette_cursor_x, roulette_cursor_y = cx, cy
1367
1368     if py16.btnp('z'):
1369         target = get_spot_from_cursor(cx, cy)
1370         if target is not None:
1371             if current_bet <= money and sum(roulette_bets.values()) + current_bet <= 5
1372             roulette_bets[target] = roulette_bets.get(target, 0) + current_bet
1373             money -= current_bet; play_sound("chip")
1374             else: play_sound("error")
1375
1376     if py16.btnp('x'):
1377         if sum(roulette_bets.values()) > 0:
1378             play_sound("blip"); state = "ROULETTE_SPIN"; timer = 120; roulette_result
1379             roulette_angle, roulette_speed = 0.0, 0.6
1380             else: play_sound("error")
1381
1382     if py16.btnp('space'): play_sound("blip"); state = "BETTING"
1383
1384     elif state == "ROULETTE_SPIN":
1385         roulette_angle += roulette_speed
1386         roulette_speed *= 0.97
1387         timer -= 1
1388         if timer % 8 == 0 and roulette_speed > 0.1: play_sound("card")
1389         if timer <= 0: evaluate_roulette()
1390
1391     elif state == "PLAYER_TURN":
1392         if game_mode == "BLACKJACK":
1393             actions = bj_available_actions()
1394             # action_idx in zulaessigen Bereich klemmen, falls Aktion verschwindet
1395             if action_idx >= len(actions): action_idx = len(actions) - 1
1396             if action_idx < 0: action_idx = 0
1397
1398             if py16.btnp('left') and action_idx > 0: action_idx -= 1; play_sound("blip")
1399             if py16.btnp('right') and action_idx < len(actions) - 1: action_idx += 1; play
1400
1401         if py16.btnp('z'):
1402             chosen = actions[action_idx]
1403             if chosen == "HIT":
1404                 play_sound("card")
1405                 bj_hands[bj_hand_idx].append(draw_card_from_deck())
1406                 player_hand = bj_hands[bj_hand_idx]
1407                 if calc_bj_score(player_hand) >= 21:
1408                     advance_bj_hand()
```

PY16CASINO - CODE LISTING (PAGE 23)

```
1409         elif chosen == "STAND":
1410             play_sound("chip")
1411             advance_bj_hand()
1412         elif chosen == "DOUBLE":
1413             play_sound("chip")
1414             bj_bets[bj_hand_idx] *= 2
1415             bj_hands[bj_hand_idx].append(draw_card_from_deck())
1416             player_hand = bj_hands[bj_hand_idx]
1417             advance_bj_hand() # nach DOUBLE genau eine Karte und dann weiter
1418         elif chosen == "SPLIT":
1419             play_sound("chip")
1420             c1 = bj_hands[0][0]
1421             c2 = bj_hands[0][1]
1422             bj_hands = [[c1, draw_card_from_deck()], [c2, draw_card_from_deck()]]
1423             bj_bets = [current_bet, current_bet]
1424             bj_hand_idx = 0
1425             player_hand = bj_hands[0]
1426             action_idx = 0
1427             unlock("BJ_SPLIT")
1428             # Falls die erste Split-Hand schon 21 hat (z.B. Ass + 10), auto-vorrue
1429             if calc_bj_score(player_hand) >= 21:
1430                 advance_bj_hand()
1431     else:
1432         # PREMIUM (unveraendert)
1433         if py16.btnp('left') and action_idx != 0: action_idx = 0; play_sound("blip")
1434         if py16.btnp('right') and action_idx != 1: action_idx = 1; play_sound("blip")
1435         if py16.btnp('z'):
1436             if action_idx == 0:
1437                 play_sound("card"); player_hand.append(draw_card_from_deck())
1438                 state = "DEALER_TURN"; timer = 30
1439             else:
1440                 play_sound("chip"); state = "DEALER_TURN"; timer = 30
1441         if len(player_hand) >= 3 and state == "PLAYER_TURN":
1442             state = "DEALER_TURN"; timer = 30
1443
1444     elif state == "DEALER_TURN":
1445         timer -= 1
1446         if timer <= 0:
1447             if game_mode == "PREMIUM":
1448                 if calc_score(dealer_hand) < 5 and len(dealer_hand) < 3:
1449                     play_sound("card"); dealer_hand.append(draw_card_from_deck()); timer =
1450                     else: evaluate_premium()
1451             elif game_mode == "BLACKJACK":
1452                 # Dealer zieht nur, wenn mind. eine Spielerhand noch nicht ueberkauft ist
1453                 any_alive = any(calc_bj_score(h) <= 21 for h in bj_hands) if bj_hands else
1454                 if any_alive and calc_bj_score(dealer_hand) < 17:
1455                     play_sound("card"); dealer_hand.append(draw_card_from_deck()); timer =
1456                     else: evaluate_blackjack()
1457
1458     elif state == "POKER_DRAW":
1459         if py16.btnp('left') and poker_cursor > 0: poker_cursor -= 1; play_sound("blip")
1460         if py16.btnp('right') and poker_cursor < 4: poker_cursor += 1; play_sound("blip")
1461         if py16.btnp('z'): poker_held[poker_cursor] = not poker_held[poker_cursor]; play_s
1462         if py16.btnp('x'):
1463             play_sound("card")
1464             for i in range(5):
1465                 if not poker_held[i]: player_hand[i] = draw_card_from_deck()
1466             state = "POKER_EVAL"; timer = 30
1467
1468     elif state == "POKER_EVAL":
1469         timer -= 1
1470         if timer <= 0: evaluate_poker()
1471
1472     elif state == "GAME_OVER":
```

PY16CASINO - CODE LISTING (PAGE 24)

```
1473     if py16.btnp('space') or py16.btnp('z'):
1474         play_sound("blip")
1475
1476     # --- Lifetime-Stats: Delta gegenueber Rundenstart auswerten ---
1477     if _money_at_round_start is not None:
1478         delta = money - _money_at_round_start
1479         if delta > 0:
1480             lifetime_won += delta
1481             if delta > biggest_win: biggest_win = delta
1482             # Win-basierte Achievements
1483             unlock("FIRST_WIN")
1484             if delta >= 500: unlock("BIG_WIN")
1485             if delta >= 2000: unlock("HUGE_WIN")
1486         elif delta < 0:
1487             lifetime_lost += -delta
1488         games_played += 1
1489         if games_played >= 100: unlock("VETERAN")
1490         _money_at_round_start = None
1491
1492     # Millionaer: aktuelles Vermoeegen
1493     if money >= 10000: unlock("MILLIONAIRE")
1494
1495     # Bankrott: Reset auf einen Notgroschen, abhaengig von der Schwierigkeit.
1496     # 25% der Start-Bankroll der gewaehlten Stufe (EASY 2500 / MEDIUM 1250 / HARD
1497     if money <= 0 and slot_free_spins == 0:
1498         unlock("BANKRUPT")
1499         comeback = 500 # Fallback
1500         for _id, _name, _bankroll, _desc in DIFFICULTIES:
1501             if _id == difficulty:
1502                 comeback = _bankroll // 4
1503                 break
1504         money = comeback
1505         msg = "BANKRUPT!"
1506         msg2 = "COMEBACK STAKE: " + str(comeback)
1507         roulette_bets.clear()
1508         bj_hands, bj_bets, bj_hand_idx = [], [], 0
1509
1510     # Stand auf Disk schreiben (Fehler werden geschluckt)
1511     save_state()
1512
1513     if game_mode == "SLOTS" and slot_free_spins > 0:
1514         state = "SLOTS_SPIN"; timer = 160
1515         slot_target_stops = [random.randint(0, len(slot_strips[c])-1) for c in ran
1516         active_coins, slot_winning_lines, slot_winning_scatters = [], [], []
1517     else:
1518         state = "BETTING"
1519
1520 # --- DRAWING HELPER ---
1521 def get_card_str(val):
1522     if val == 1 or val == 14: return "A"
1523     if val == 11: return "J"
1524     if val == 12: return "Q"
1525     if val == 13: return "K"
1526     return str(val)
1527
1528 def draw_suit_icon(cx, cy, suit):
1529     color = 8 if suit in (1, 3) else 5
1530     if suit == 0: # Pik (Spade) - schwarz
1531         py16.pset(cx+1, cy, color); py16.rectfill(cx, cy+1, 3, 2, color); py16.pset(cx+1,
1532     elif suit == 1: # Herz (Heart) - rot
1533         py16.pset(cx, cy, color); py16.pset(cx+2, cy, color); py16.rectfill(cx, cy+1, 3, 2
1534     elif suit == 2: # Kreuz (Club) - schwarz: drei Blaetter + Stiel
1535         py16.pset(cx+1, cy, color)
1536         py16.pset(cx, cy+1, color); py16.pset(cx+2, cy+1, color)
```

PY16CASINO - CODE LISTING (PAGE 25)

```
1537         py16.pset(cx+1, cy+2, color); py16.pset(cx+1, cy+3, color)
1538     elif suit == 3: # Karo (Diamond) - rot: schlanker Rhombus
1539         py16.pset(cx+1, cy, color)
1540         py16.pset(cx, cy+1, color); py16.pset(cx+2, cy+1, color)
1541         py16.pset(cx+1, cy+2, color)
1542
1543 def draw_card(cx, cy, card_data, hidden=False):
1544     if hidden:
1545         py16.rectfill(cx, cy, 24, 34, 8); py16.rect(cx, cy, 24, 34, 0); py16.rectfill(cx+2
1546     else:
1547         py16.rectfill(cx, cy, 24, 34, 7); py16.rect(cx, cy, 24, 34, 0)
1548         val, suit = card_data
1549         s = get_card_str(val); tw = len(s) * 4 - 1; text_color = 8 if suit in (1, 3) else
1550         py16.text(s, cx + 2, cy + 2, text_color); draw_suit_icon(cx + 4 + tw, cy + 2, suit
1551         tx, ty = cx + 12 - tw // 2, cy + 14
1552         py16.text(s, tx, ty, text_color); py16.text(s, tx + 1, ty, text_color)
1553
1554 def draw_chip(cx, cy, val, color, is_selected, size_mod=0):
1555     bw = len(str(val)) * 4; radius = 14 + size_mod
1556     if is_selected: py16.circfill(cx, cy, radius + 2, 10)
1557     else: py16.circfill(cx, cy, radius, 0)
1558     py16.circfill(cx, cy, radius - 1, color); py16.circ(cx, cy, radius - 4, 7); py16.circf
1559     py16.text(str(val), cx - bw//2 + 1, cy - 2, 7)
1560
1561 def draw_slot_symbol(cx, cy, sym):
1562     if sym == "KIR":
1563         py16.line(cx-3, cy+1, cx+1, cy-5, 11); py16.line(cx+4, cy+1, cx+1, cy-5, 11)
1564         py16.rectfill(cx+1, cy-6, 3, 2, 11); py16.circfill(cx-3, cy+3, 3, 8)
1565         py16.circfill(cx+4, cy+3, 3, 8); py16.pset(cx-4, cy+2, 7); py16.pset(cx+3, cy+2, 7
1566     elif sym == "ZIT":
1567         py16.circfill(cx, cy, 4, 10); py16.circfill(cx-2, cy, 3, 10); py16.circfill(cx+2,
1568         py16.pset(cx-5, cy, 10); py16.pset(cx+5, cy, 10); py16.rectfill(cx-1, cy-5, 2, 2,
1569     elif sym == "ORG":
1570         py16.circfill(cx, cy+1, 5, 9); py16.rectfill(cx, cy-4, 2, 2, 11); py16.pset(cx-2,
1571     elif sym == "PFL":
1572         py16.circfill(cx, cy+1, 4, 2); py16.circfill(cx-1, cy+2, 3, 2); py16.circfill(cx+1
1573         py16.rectfill(cx, cy-4, 2, 2, 11); py16.pset(cx-2, cy, 14)
1574     elif sym == "GLO":
1575         py16.circfill(cx, cy-1, 3, 10); py16.rectfill(cx-4, cy-1, 9, 5, 10)
1576         py16.rectfill(cx-5, cy+3, 11, 2, 10); py16.circfill(cx, cy+5, 1, 9); py16.line(cx-
1577     elif sym == "DIA":
1578         py16.rectfill(cx-3, cy-3, 7, 2, 6); py16.line(cx-5, cy-1, cx+5, cy-1, 7); py16.lin
1579         py16.line(cx-3, cy+1, cx+3, cy+1, 12); py16.line(cx-2, cy+2, cx+2, cy+2, 12)
1580         py16.line(cx-1, cy+3, cx+1, cy+3, 12); py16.pset(cx, cy+4, 12); py16.pset(cx-2, cy
1581     elif sym == "7":
1582         py16.rectfill(cx-4, cy-4, 9, 2, 8); py16.rectfill(cx+3, cy-2, 2, 2, 8); py16.rectf
1583         py16.rectfill(cx-1, cy+2, 2, 2, 8); py16.rectfill(cx-3, cy+4, 2, 2, 8); py16.line(
1584     elif sym == "BNS":
1585         py16.circfill(cx, cy, 6, 10); py16.circ(cx, cy, 6, 9); py16.text("B", cx-1, cy-2,
1586
1587 # Gebäude zeichnen mit Unterstützung für Nacht und Tag Modus
1588 def draw_bg_building(bx, by, bw, bh, bcol, is_office, is_night):
1589     py16.rectfill(bx, by, bw, bh, bcol)
1590     for wy in range(by + 6, by + bh, 12):
1591         for wx in range(bx + 4, bx + bw - 6, 8):
1592             val = (wx * 17 + wy * 31) % 10
1593             if val > 6:
1594                 if is_night:
1595                     col = 7 if is_office else 9
1596                     if val == 9: col = 6 if is_office else 10
1597                 else:
1598                     col = 12 if is_office else 5 # Tagsüber reflektierende oder dunkle Fen
1599                 py16.rectfill(wx, wy, 3, 5, col)
1600             elif val > 2:
```

PY16CASINO - CODE LISTING (PAGE 26)

```
1601         py16.rectfill(wx, wy, 3, 5, 0 if bcol != 0 else 1)
1602
1603 # --- MAIN DRAW ---
1604 def draw():
1605     # Tag/Nacht Status auch beim Zeichnen aktualisieren
1606     hour = datetime.datetime.now().hour
1607     is_night = (hour >= 20 or hour < 8)
1608
1609     # =====
1610     # --- SCHWIERIGKEITSWAHL (nur beim allerersten Start) ---
1611     # =====
1612     if state == "DIFFICULTY_SELECT":
1613         t = py16.t()
1614         py16.cls(1) # Dunkelblau
1615         py16.rect(0, 0, 256, 224, 4)
1616         py16.rect(1, 1, 254, 222, 10)
1617
1618         # Titel
1619         title = tr("diff_welcome")
1620         py16.text(title, 128 - len(title)*4//2, 28, 10)
1621         sub = tr("diff_choose")
1622         py16.text(sub, 128 - len(sub)*4//2, 42, 7)
1623         py16.line(40, 54, 216, 54, 5)
1624
1625         # Schwierigkeits-Beschreibungen je nach Stufe (uebersetzbar)
1626         _diff_desc_keys = {"EASY": "diff_desc_easy", "MEDIUM": "diff_desc_medium", "HARD":
1627
1628         # Drei Schwierigkeits-Karten untereinander
1629         card_w, card_h = 180, 38
1630         card_x = 128 - card_w // 2
1631         first_y = 68
1632         gap = 8
1633         for i, (diff_id, name, bankroll, desc) in enumerate(DIFFICULTIES):
1634             cy = first_y + i * (card_h + gap)
1635             selected = (i == diff_cursor)
1636
1637             # Karten-Hintergrund + Rahmen
1638             if selected:
1639                 py16.rectfill(card_x, cy, card_w, card_h, 5)
1640                 # Pulsierender Goldrahmen
1641                 rcol = 10 if (t // 8) % 2 == 0 else 9
1642                 py16.rect(card_x, cy, card_w, card_h, rcol)
1643                 py16.rect(card_x - 1, cy - 1, card_w + 2, card_h + 2, rcol)
1644             else:
1645                 py16.rectfill(card_x, cy, card_w, card_h, 0)
1646                 py16.rect(card_x, cy, card_w, card_h, 5)
1647
1648             # Auswahl-Pfeil links
1649             if selected:
1650                 ax = card_x - 10
1651                 ay = cy + card_h // 2
1652                 py16.text(">", ax, ay - 3, 10)
1653
1654             # Name (gross, links) - EASY/MEDIUM/HARD bleiben sprachneutral
1655             name_col = 10 if selected else 7
1656             py16.text(name, card_x + 10, cy + 7, name_col)
1657
1658             # Bankroll (rechts, gross)
1659             br_str = "$" + str(bankroll)
1660             br_col = 11 if selected else 6
1661             py16.text(br_str, card_x + card_w - len(br_str)*4 - 10, cy + 7, br_col)
1662
1663             # Beschreibung (klein, darunter) - uebersetzt
1664             desc_col = 7 if selected else 5
```

PY16CASINO - CODE LISTING (PAGE 27)

```
1665         desc_txt = tr(_diff_desc_keys.get(diff_id, "diff_desc_medium"))
1666         py16.text(desc_txt, card_x + 10, cy + 22, desc_col)
1667
1668         # "STARTING BANKROLL"-Label klein unter Bankroll - uebersetzt
1669         lbl = tr("diff_bankroll")
1670         py16.text(lbl, card_x + card_w - len(lbl)*4 - 10, cy + 22, desc_col)
1671
1672         # Help-Text unten
1673         help_txt = tr("diff_help")
1674         if (t // 30) % 2 == 0:
1675             py16.text(help_txt, 128 - len(help_txt)*4//2, 200, 8)
1676         note = tr("diff_once")
1677         py16.text(note, 128 - len(note)*4//2, 212, 5)
1678         return
1679
1680     # =====
1681     # --- MENUE (CASINO SKYLINE MIT WASSERREFLEXION & TAG/NACHT-ZYKLUS) ---
1682     # =====
1683     if state == "MENU":
1684         t = py16.t()
1685
1686         # --- HIMMEL & HINTERGRUND ---
1687         if is_night:
1688             py16.cls(1) # Dunkelblau bei Nacht
1689
1690             # Sterne (Cache, kein globaler seed-Eingriff mehr)
1691             for sx, sy, sc, twink in star_cache:
1692                 col = sc
1693                 if twink and (t//10) % 2 == 0: col = 1
1694                 py16.pset(sx, sy, col)
1695
1696             # Mond mit additivem Glow
1697             py16.circfill(210, 40, 18, 10)
1698             py16.blend_mode("add")
1699             py16.circfill(210, 40, 22, 9)
1700             py16.blend_mode("normal")
1701             py16.circfill(204, 36, 16, 1)
1702
1703             # Partikel (Feuerwerk)
1704             py16.particles_draw()
1705
1706             # Suchscheinwerfer (Searchlights)
1707             py16.blend_mode("add")
1708             for i in range(3):
1709                 angle = math.sin(t * 0.015 + i * 2.5) * 0.6 - 1.57
1710                 bx, by = 128 + (i-1)*60, 170
1711                 ex, ey = bx + math.cos(angle)*200, by + math.sin(angle)*200
1712                 py16.line(int(bx), int(by), int(ex), int(ey), 12)
1713                 py16.line(int(bx)-1, int(by), int(ex)-1, int(ey), 1)
1714                 py16.line(int(bx)+1, int(by), int(ex)+1, int(ey), 1)
1715             py16.blend_mode("normal")
1716
1717         else:
1718             py16.cls(12) # Hellblau am Tag
1719
1720             # Flauschige Wolken zeichnen
1721             py16.circfill(40, 30, 12, 7)
1722             py16.circfill(55, 25, 16, 7)
1723             py16.circfill(70, 32, 14, 7)
1724
1725             py16.circfill(180, 50, 10, 7)
1726             py16.circfill(195, 45, 15, 7)
1727             py16.circfill(210, 52, 12, 7)
1728
```

PY16CASINO - CODE LISTING (PAGE 28)

```
1729         # Sonne
1730         py16.circfill(210, 40, 18, 10)
1731
1732         # --- SKYLINE (Wird bis ganz nach unten gezeichnet für die Reflexion) ---
1733         draw_bg_building(10, 110, 40, 114, 2, True, is_night) # Links lila
1734         draw_bg_building(35, 70, 30, 154, 0, False, is_night) # Links schwarz
1735
1736         draw_bg_building(190, 80, 45, 144, 2, False, is_night) # Rechts lila
1737         draw_bg_building(215, 110, 35, 114, 0, True, is_night) # Rechts schwarz
1738
1739         # Haupt-Casino Hotel
1740         py16.rectfill(74, 50, 108, 174, 5)
1741         py16.rect(74, 50, 108, 174, 0)
1742
1743         # Leuchtende Fenster Hauptgebäude
1744         for wy in range(60, 224, 12):
1745             for wx in range(80, 176, 10):
1746                 val = (wx * 93 + wy * 57) % 100
1747                 if val > 25:
1748                     if is_night:
1749                         col = 10 if val > 80 else 9
1750                         if val % 11 == 0 and (t // (4 + wx%4)) % 2 == 0: col = 1
1751                     else:
1752                         col = 1 # Fenster aus bei Tag
1753                     py16.rectfill(wx, wy, 4, 6, col)
1754                 else:
1755                     py16.rectfill(wx, wy, 4, 6, 1)
1756
1757         # =====
1758         # --- WASSER REFLEXIONS-EFFEKT (SCANLINES) ---
1759         # =====
1760         water_y = 166 # 1mm weiter unten, damit es glatt mit dem Fenster am zweiten Gebaeu
1761
1762         # Wasser abdunkeln/einfärben
1763         if is_night:
1764             py16.blend_mode("alpha", alpha=140)
1765             py16.rectfill(0, water_y, 256, 224 - water_y, 1) # Dunkelblauer Tint ueber die
1766         else:
1767             py16.blend_mode("alpha", alpha=100)
1768             py16.rectfill(0, water_y, 256, 224 - water_y, 12) # Hellblauer Tint bei Tag
1769             py16.blend_mode("normal")
1770
1771         # Scanline-Verzerrung
1772         wave = py16.scanline_wave(t, amplitude=4, frequency=0.08, speed=2, y_start=water_y)
1773         py16.scanline_apply(wave, wrap=False, fill_color=1 if is_night else 12)
1774
1775         # Uferlinie / Horizont einzeichnen
1776         py16.line(0, water_y, 256, water_y, 12 if is_night else 7)
1777         py16.line(0, water_y+1, 256, water_y+1, 1 if is_night else 6)
1778
1779         # =====
1780         # --- UI & LOGO (Schweben SCHARF über dem Wasser) ---
1781         # =====
1782         # --- Logo Animation ---
1783         scale = 4
1784         hover_y = int(math.sin(t * 0.05) * 4)
1785         sh_x = 2 + int(math.cos(t * 0.03) * 3)
1786         sh_y = 2 + int(math.sin(t * 0.04) * 3)
1787
1788         py16_x = 128 - (len(logo_py16[0]) * scale) // 2
1789         py16_y = 20 + hover_y
1790         cas_x = 128 - (len(logo_casino[0]) * scale) // 2
1791         cas_y = py16_y + len(logo_py16) * scale + 8
1792
```

PY16CASINO - CODE LISTING (PAGE 29)

```
1793     # Glow Layer (Nur in der Nacht)
1794     if is_night:
1795         py16.blend_mode("add")
1796         for r, row in enumerate(logo_py16):
1797             for c, char in enumerate(row):
1798                 if char != ' ': py16.rectfill(py16_x + c*scale - 1, py16_y + r*scale -
1799         for r, row in enumerate(logo_casino):
1800             for c, char in enumerate(row):
1801                 if char != ' ': py16.rectfill(cas_x + c*scale - 1, cas_y + r*scale - 1
1802         py16.blend_mode("normal")
1803
1804     # Schatten und Main Text Layer
1805     for layer in range(2):
1806         ox = sh_x if layer == 0 else 0
1807         oy = sh_y if layer == 0 else 0
1808
1809         for r, row in enumerate(logo_py16):
1810             for c, char in enumerate(row):
1811                 if char != ' ':
1812                     col = 0 if layer == 0 else 10
1813                     if layer == 1 and abs(c - (t//3) % (len(row)+10)) < 2: col = 7
1814                     py16.rectfill(py16_x + c*scale + ox, py16_y + r*scale + oy, scale,
1815
1816         for r, row in enumerate(logo_casino):
1817             for c, char in enumerate(row):
1818                 if char != ' ':
1819                     col = 0 if layer == 0 else 7
1820                     if layer == 1 and abs(c - (t//3 + 5) % (len(row)+10)) < 2: col = 6
1821                     py16.rectfill(cas_x + c*scale + ox, cas_y + r*scale + oy, scale, s
1822
1823     # --- Zentrierte Menue-Punkte in Box ---
1824     box_y = 122
1825
1826     # Box Glow (Nur nachts)
1827     if is_night:
1828         py16.blend_mode("add")
1829         py16.rect(62, box_y-2, 132, 70, 2)
1830         py16.blend_mode("normal")
1831
1832     py16.rectfill(64, box_y, 128, 66, 0)
1833     py16.rect(64, box_y, 128, 66, 14 if is_night else 5) # Pink nachts, Dunkelgrau tag
1834
1835     sound_txt = tr("on") if sound_enabled else tr("off")
1836     ach_txt = f"{tr('lbl_achievements')} ({len(unlocked_achievements)}/{ACH_TOTAL})"
1837     lang_name = next((l[1] for l in LANGUAGES if l[0] == language), language)
1838
1839     if not game_started:
1840         items = [
1841             tr("lbl_game") + ": " + game_mode,
1842             tr("menu_start"),
1843             tr("lbl_sound") + ": " + sound_txt,
1844             ach_txt,
1845             tr("lbl_language") + ": " + lang_name,
1846         ]
1847     else:
1848         items = [
1849             tr("menu_resume"),
1850             tr("menu_end"),
1851             tr("lbl_sound") + ": " + sound_txt,
1852             ach_txt,
1853             tr("lbl_language") + ": " + lang_name,
1854         ]
1855
1856     for i, item in enumerate(items):
```

PY16CASINO - CODE LISTING (PAGE 30)

```
1857         color = 10 if i == menu_idx else 6
1858         # ACHIEVEMENTS bei 100% in Gold faerben
1859         if i == 3 and len(unlocked_achievements) == ACH_TOTAL and i != menu_idx:
1860             color = 10
1861
1862         # GAME (idx 0, nur vor Spielstart) und LANGUAGE (idx 4) zeigen < > Cycle-Pfeil
1863         if i == 0 and not game_started:
1864             text_str = f"{tr('lbl_game')}: < {game_mode} >"
1865         elif i == 4:
1866             text_str = f"{tr('lbl_language')}: < {lang_name} >"
1867         else:
1868             text_str = item
1869
1870         if i == menu_idx: text_str = f"> {text_str} <"
1871
1872         tw = len(text_str) * 4
1873         x = 128 - tw // 2
1874         y = box_y + 8 + i * 12 # Spacing 12, damit 5 Eintraege in die Box passen
1875
1876         py16.text(text_str, x, y, color)
1877
1878         help_txt = tr("menu_help")
1879         py16.text(help_txt, 128 - (len(help_txt)*4)//2, box_y + 74, 5)
1880
1881         # --- Lifetime-Stats (nur wenn es bereits gespielte Runden gibt) ---
1882         # Hinweis: Achievement-Counter erscheint jetzt direkt im Menue-Eintrag,
1883         # daher hier nur noch GAMES/NET und BIGGEST WIN.
1884         if games_played > 0:
1885             stats_y = box_y + 86
1886             net = lifetime_won - lifetime_lost
1887             sign = "+" if net >= 0 else "-"
1888             line1 = f"{tr('stat_games')}: {games_played} {tr('stat_net')}: {sign}${abs(n
1889             py16.text(line1, 128 - (len(line1)*4)//2, stats_y, 9)
1890             line2 = f"{tr('stat_biggest')}: ${biggest_win}"
1891             py16.text(line2, 128 - (len(line2)*4)//2, stats_y + 8, 9)
1892
1893         return
1894
1895         # =====
1896         # --- ACHIEVEMENT-LISTE (eigener Screen) ---
1897         # =====
1898         if state == "ACHIEVEMENTS":
1899             py16.cls(1) # Dunkelblauer Hintergrund
1900             py16.rect(0, 0, 256, 224, 4); py16.rect(1, 1, 254, 222, 10)
1901
1902             # Titel
1903             n_unlocked = len(unlocked_achievements)
1904             title = f"{tr('lbl_achievements')} {n_unlocked} / {ACH_TOTAL}"
1905             title_col = 10 if n_unlocked == ACH_TOTAL else 7
1906             tw = len(title) * 4
1907             py16.text(title, 128 - tw // 2, 10, title_col)
1908             py16.line(20, 22, 236, 22, 5)
1909
1910             # Zwei-Spalten-Liste
1911             col_size = (ACH_TOTAL + 1) // 2
1912             line_h = 11
1913             start_y = 30
1914             for idx, ach in enumerate(ACHIEVEMENTS):
1915                 ach_id = ach[0]
1916                 loc_name, _loc_desc = tr_ach(ach_id)
1917                 unlocked = ach_id in unlocked_achievements
1918                 col = idx // col_size
1919                 row = idx % col_size
1920                 x = 10 + col * 124
```

PY16CASINO - CODE LISTING (PAGE 31)

```
1921         y = start_y + row * line_h
1922
1923         # Cursor-Highlight
1924         if idx == ach_cursor:
1925             py16.rectfill(x - 2, y - 1, 122, line_h, 5)
1926
1927         # Icon: gefuellter Stern bei unlocked, kleines Schloss bei locked
1928         if unlocked:
1929             icon_col = 10
1930             # Mini-Stern 7x7
1931             ix, iy = x, y + 1
1932             py16.pset(ix + 3, iy, icon_col)
1933             py16.pset(ix + 3, iy + 1, icon_col); py16.pset(ix + 2, iy + 2, icon_col);
1934             py16.rectfill(ix, iy + 3, 7, 1, icon_col)
1935             py16.pset(ix + 1, iy + 4, icon_col); py16.pset(ix + 5, iy + 4, icon_col)
1936             py16.pset(ix + 2, iy + 5, icon_col); py16.pset(ix + 4, iy + 5, icon_col)
1937         else:
1938             # Mini-Schloss 7x7
1939             lock_col = 5
1940             ix, iy = x, y + 1
1941             py16.rectfill(ix + 1, iy, 5, 3, lock_col) # Buegel oben
1942             py16.rectfill(ix + 2, iy + 1, 3, 2, 1) # Buegel-Inneres ausschneiden
1943             py16.rectfill(ix, iy + 3, 7, 4, lock_col) # Korpus
1944             py16.pset(ix + 3, iy + 4, 1) # Schluesselloch
1945
1946         # Name (uebersetzt, auf 18 Zeichen begrenzt)
1947         name_col = 7 if unlocked else 6
1948         py16.text(loc_name[:18], x + 10, y + 2, name_col)
1949
1950         # Beschreibung des aktuell ausgewaehlten Achievements unten anzeigen
1951         sel = ACHIEVEMENTS[ach_cursor]
1952         sel_id = sel[0]
1953         sel_name, sel_desc = tr_ach(sel_id)
1954         sel_unlocked = sel_id in unlocked_achievements
1955         box_top = 184
1956         py16.rectfill(8, box_top, 240, 28, 0)
1957         py16.rect(8, box_top, 240, 28, 10 if sel_unlocked else 5)
1958         py16.text(sel_name, 14, box_top + 4, 10 if sel_unlocked else 6)
1959         # Status-Label rechts
1960         status = tr("ach_unlocked") if sel_unlocked else tr("ach_locked")
1961         status_col = 10 if sel_unlocked else 5
1962         sw = len(status) * 4
1963         py16.text(status, 244 - sw - 4, box_top + 4, status_col)
1964         py16.text(sel_desc, 14, box_top + 14, 7)
1965
1966         # Help-Text ganz unten
1967         help_txt = tr("ach_help")
1968         py16.text(help_txt, 128 - (len(help_txt)*4)//2, 216, 6)
1969         return
1970
1971         # =====
1972         # --- SPIELTISCHE ---
1973         # =====
1974         py16.cls(3)
1975         py16.rect(0, 0, 256, 224, 4); py16.rect(1, 1, 254, 222, 9); py16.rect(2, 2, 252, 220,
1976
1977         # --- HUD ---
1978         py16.rectfill(0, 0, 256, 14, 0)
1979         py16.text("JACKPOT: " + str(jackpot), 4, 4, 10); py16.text("MONEY: " + str(money), 140
1980         py16.text("ENTER: MENU", 204, 4, 6); py16.line(0, 14, 256, 14, 4)
1981
1982         # --- SLOTS TISCH ---
1983         if game_mode == "SLOTS":
1984             py16.rectfill(16, 16, 224, 154, 6); py16.rect(16, 16, 224, 154, 5); py16.rectfill(
```

PY16CASINO - CODE LISTING (PAGE 32)

```
1985     py16.rectfill(30, 24, 196, 16, 0); py16.rect(29, 23, 198, 18, 1)
1986     if state == "GAME_OVER":
1987         txt = msg + " " + msg2; py16.text(txt, 128 - (len(txt)*4)//2, 29, 10)
1988     elif state == "SLOTS_SPIN":
1989         if slot_free_spins > 0: py16.text(f"FREE SPINS: {slot_free_spins}", 96, 29, 11)
1990         else: py16.text("GOOD LUCK!", 108, 29, 11)
1991     else:
1992         if slot_free_spins > 0: py16.text(f"FREE SPINS: {slot_free_spins}", 96, 29, 11)
1993         else: py16.text("INSERT COIN TO PLAY", 90, 29, 8)
1994
1995     py16.rectfill(30, 44, 196, 102, 0); py16.clip(30, 44, 196, 102)
1996     for c in range(5):
1997         off_y = slot_pixel_offset[c]
1998         base_idx = slot_current_idx[c]
1999         for r in range(-1, 3):
2000             idx = (base_idx + r) % len(slot_strips[c])
2001             sym = slot_strips[c][idx]
2002             x = 34 + c * 38; y = 44 + r * 34 + int(off_y)
2003             py16.rectfill(x, y, 34, 30, 7); py16.rect(x, y, 34, 30, 6)
2004             if state in ["BETTING", "SLOTS_SPIN", "GAME_OVER", "SLOTS_PAYTABLE"]: draw
2005     py16.clip()
2006
2007     if state == "GAME_OVER":
2008         blink_col = 10 if (py16.t() // 15) % 2 == 0 else 8
2009         for r, c in slot_winning_scatters:
2010             x, y = 34 + c * 38, 44 + r * 34
2011             py16.rect(x, y, 34, 30, blink_col); py16.rect(x+1, y+1, 32, 28, blink_col)
2012         for line in slot_winning_lines:
2013             for i in range(4):
2014                 r1, c1 = line[i]; r2, c2 = line[i+1]
2015                 x1, y1 = 34 + c1 * 38 + 17, 44 + r1 * 34 + 15; x2, y2 = 34 + c2 * 38 +
2016                 py16.line(x1, y1, x2, y2, blink_col); py16.line(x1, y1-1, x2, y2-1, bl
2017
2018     py16.rectfill(98, 152, 60, 12, 0); py16.rect(98, 152, 60, 12, 1); py16.line(100, 1
2019
2020     if state == "SLOTS_PAYTABLE":
2021         py16.rectfill(20, 20, 216, 146, 0); py16.rect(20, 20, 216, 146, 5); py16.text(
2022         for i, sym in enumerate(["KIR", "ZIT", "ORG", "PFL"]):
2023             cy = 40 + i * 28; draw_slot_symbol(36, cy+4, sym); p = slot_payouts[sym]
2024             py16.text("3X:{} 4X:{} 5X:{}".format(p[3], p[4], p[5]), 50, cy+2, 7)
2025         for i, sym in enumerate(["GLO", "DIA", " " 7 "]):
2026             cy = 40 + i * 28; draw_slot_symbol(136, cy+4, sym); p = slot_payouts[sym]
2027             py16.text("3X:{} 4X:{} 5X:{}".format(p[3], p[4], p[5]), 150, cy+2, 7)
2028             cy = 40 + 3 * 28; draw_slot_symbol(136, cy+4, "BNS"); py16.text("3+ SCATTER=10
2029             py16.text("PRESS Z TO RETURN", 76, 154, 8)
2030
2031     # --- POKER TISCH (AUTOMATEN-GEHAEUSE) ---
2032     elif game_mode == "POKER":
2033         py16.rectfill(16, 16, 224, 154, 6); py16.rect(16, 16, 224, 154, 5)
2034         py16.rectfill(30, 24, 196, 16, 0); py16.rect(29, 23, 198, 18, 1)
2035         if state == "GAME_OVER":
2036             txt = msg + " " + msg2; py16.text(txt, 128 - (len(txt)*4)//2, 29, 10)
2037         elif state == "POKER_DRAW": py16.text("Z: HOLD  X: DRAW", 84, 29, 11)
2038         else: py16.text("INSERT COIN TO PLAY", 90, 29, 8)
2039
2040     py16.rectfill(26, 44, 204, 102, 1); py16.rect(25, 43, 206, 104, 5)
2041
2042     if state == "POKER_PAYTABLE":
2043         py16.rectfill(30, 48, 196, 94, 0); py16.rect(30, 48, 196, 94, 5)
2044         py16.text("--- POKER PAYTABLE ---", 68, 54, 10)
2045         py16.text("ROYAL FLUSH:  JACKPOT", 36, 70, 11); py16.text("STR. FLUSH:
2046         py16.text("4 OF A KIND:      25X", 36, 94, 7); py16.text("FULL HOUSE:
2047         py16.text("FLUSH:           6X", 36, 118, 7); py16.text("STRAIGHT:
2048         py16.text("3 OF A KIND:     3X", 140, 82, 7); py16.text("TWO PAIR:
```

PY16CASINO - CODE LISTING (PAGE 33)

```
2049         py16.text("JACKS OR BETTER:    1X", 140, 106, 7); py16.text("PRESS Z TO RETURN
2050     else:
2051         for i in range(5):
2052             cx = 36 + i * 38; cy = 70
2053             if state == "BETTING": py16.rect(cx, cy, 24, 34, 5)
2054             else:
2055                 if i < len(player_hand): draw_card(cx, cy, player_hand[i], False)
2056                 if state == "POKER_DRAW":
2057                     if poker_held[i]: py16.rectfill(cx-2, cy-12, 28, 10, 8); py16.text
2058                     if i == poker_cursor:
2059                         py16.rect(cx-2, cy-2, 28, 38, 10); py16.rect(cx-1, cy-1, 26, 3
2060
2061         py16.rectfill(98, 152, 60, 12, 0); py16.rect(98, 152, 60, 12, 1); py16.line(100, 1
2062
2063     # --- ROULETTE TISCH ---
2064     elif game_mode == "ROULETTE":
2065         targets = ["0"] + [str(i) for i in range(1,37)] + ["COL1", "COL2", "COL3", "1ST12", "
2066         for t in targets:
2067             x, y, w, h = get_target_rect(t)
2068             col = get_roulette_color(t)
2069             py16.rectfill(x, y, w, h, col); py16.rect(x, y, w, h, 0)
2070
2071             s = "3:1" if t.startswith("COL") else t
2072             if t == "1ST12": s = "1ST 12"
2073             if t == "2ND12": s = "2ND 12"
2074             if t == "3RD12": s = "3RD 12"
2075             tw = len(s) * 4 - 1
2076             if t == "0": py16.text("0", x + w//2 - 1, y + 21, 7)
2077             else: py16.text(s, x + w//2 - tw//2, y + h//2 - 2, 7)
2078
2079         for target, amt in roulette_bets.items():
2080             tx, ty, tw, th = get_target_rect(target)
2081             if tw > 0:
2082                 c_col = 10
2083                 if amt >= 250: c_col = 2
2084                 elif amt >= 100: c_col = 11
2085                 elif amt >= 50: c_col = 8
2086                 else: c_col = 12
2087                 draw_chip(tx + tw//2, ty + th//2, amt, c_col, False, -6)
2088
2089         if state == "GAME_OVER":
2090             dx, dy, dw, dh = get_target_rect(str(roulette_result))
2091             if dw > 0:
2092                 mx, my = dx + dw//2, dy + dh//2
2093                 py16.circfill(mx, my+2, 5, 12); py16.circ(mx, my+2, 5, 0)
2094                 py16.rectfill(mx-1, my-4, 2, 6, 12); py16.circfill(mx, my-5, 3, 12); py16.
2095
2096     # --- History-Leiste der letzten Spins (neueste links) ---
2097     if roulette_history:
2098         hist_y = 105
2099         box_w, box_h, gap = 14, 12, 1
2100         py16.text("LAST:", 6, hist_y + 3, 6)
2101         bx = 32
2102         for i, num in enumerate(roulette_history):
2103             rx = bx + i * (box_w + gap)
2104             if rx + box_w > 250: break # nicht ueber den Rand zeichnen
2105             col = 11 if num == 0 else (8 if num in RED_NUMS else 5)
2106             py16.rectfill(rx, hist_y, box_w, box_h, col)
2107             py16.rect(rx, hist_y, box_w, box_h, 0)
2108             # Den neuesten Eintrag (i==0) mit hellem Rahmen pulsieren lassen
2109             if i == 0 and (py16.t() // 8) % 2 == 0:
2110                 py16.rect(rx - 1, hist_y - 1, box_w + 2, box_h + 2, 10)
2111             s = str(num)
2112             tw = len(s) * 4 - 1
```

PY16CASINO - CODE LISTING (PAGE 34)

```
2113         py16.text(s, rx + box_w//2 - tw//2, hist_y + 3, 7)
2114
2115     k_x, k_y = 128, 168
2116     py16.circfill(k_x, k_y, 46, 4); py16.circfill(k_x, k_y, 44, 9); py16.circfill(k_x,
2117     for i in range(37):
2118         ang = wheel_angle + i * (math.pi * 2 / 37)
2119         num = roulette_sequence[i]
2120         col = 11 if num == 0 else (8 if num in RED_NUMS else 5)
2121         for step in range(4):
2122             a = ang + step * 0.03
2123             px1 = k_x + math.cos(a) * 28; py1 = k_y + math.sin(a) * 28
2124             px2 = k_x + math.cos(a) * 38; py2 = k_y + math.sin(a) * 38
2125             py16.line(int(px1), int(py1), int(px2), int(py2), col)
2126             sx1 = k_x + math.cos(ang - 0.02) * 28; sy1 = k_y + math.sin(ang - 0.02) * 28
2127             sx2 = k_x + math.cos(ang - 0.02) * 38; sy2 = k_y + math.sin(ang - 0.02) * 38
2128             py16.line(int(sx1), int(sy1), int(sx2), int(sy2), 7)
2129
2130     py16.circfill(k_x, k_y, 28, 4); py16.circfill(k_x, k_y, 14, 9); py16.circ(k_x, k_y
2131     for i in range(4):
2132         ca = wheel_angle + i * (math.pi/2)
2133         py16.line(k_x, k_y, int(k_x + math.cos(ca)*14), int(k_y + math.sin(ca)*14), 10
2134         py16.line(k_x, k_y-1, int(k_x + math.cos(ca)*14), int(k_y + math.sin(ca)*14)-1
2135
2136     if state == "ROULETTE_SPIN":
2137         if timer < 60:
2138             ball_radius = 32 + 8 * (timer / 60.0)
2139             if timer < 20:
2140                 target_idx = roulette_sequence.index(roulette_result)
2141                 b_ang = wheel_angle + target_idx * (math.pi * 2 / 37) + 0.06
2142             else: b_ang = roulette_angle
2143         else:
2144             ball_radius = 40; b_ang = roulette_angle
2145     else:
2146         ball_radius = 32
2147         target_idx = roulette_sequence.index(roulette_result)
2148         b_ang = wheel_angle + target_idx * (math.pi * 2 / 37) + 0.06
2149
2150     ball_x = k_x + math.cos(b_ang) * ball_radius; ball_y = k_y + math.sin(b_ang) * bal
2151     py16.circfill(int(ball_x), int(ball_y), 3, 7)
2152
2153     # --- PREMIUM & BLACKJACK TISCH ---
2154     elif game_mode in ["PREMIUM", "BLACKJACK"]:
2155         spacing = 34 if game_mode == "PREMIUM" else 20
2156         start_x = 82
2157         is_split = (game_mode == "BLACKJACK" and len(bj_hands) > 1)
2158
2159     py16.text("DEALER:", start_x, 26, 10)
2160     if not is_split: py16.text("PLAYER:", start_x, 86, 10)
2161
2162     if state == "BETTING":
2163         slots_count = 3 if game_mode == "PREMIUM" else 2
2164         for i in range(slots_count):
2165             py16.rect(start_x + i * spacing, 36, 24, 34, 2); py16.rect(start_x + i * s
2166     else:
2167         for i, c in enumerate(dealer_hand):
2168             is_hidden = (state == "PLAYER_TURN" and i == 1)
2169             draw_card(start_x + i * spacing, 36, c, is_hidden)
2170
2171     if is_split:
2172         # Zwei Haende nebeneinander
2173         small_spacing = 14
2174         for h_idx, hand in enumerate(bj_hands):
2175             hx_base = 22 if h_idx == 0 else 142
2176             # Aktive Hand hervorheben
```

PY16CASINO - CODE LISTING (PAGE 35)

```
2177         if h_idx == bj_hand_idx and state == "PLAYER_TURN":
2178             py16.rect(hx_base - 4, 92, 100, 46, 10)
2179             py16.rect(hx_base - 3, 93, 98, 44, 10)
2180             for i, c in enumerate(hand):
2181                 draw_card(hx_base + i * small_spacing, 96, c, False)
2182             # Per-Hand-Score und Einsatz
2183             score = calc_bj_score(hand)
2184             label = "H{:}: {} (${})".format(h_idx + 1, score, bj_bets[h_idx])
2185             py16.rectfill(hx_base, 144, len(label) * 4 + 4, 8, 0)
2186             py16.text(label, hx_base + 2, 146, 11)
2187         else:
2188             # Einzelne Hand (PREMIUM oder Blackjack ohne Split)
2189             ref_hand = bj_hands[0] if (game_mode == "BLACKJACK" and bj_hands) else pla
2190             for i, c in enumerate(ref_hand): draw_card(start_x + i * spacing, 96, c, F
2191
2192             score_val = calc_score(ref_hand) if game_mode == "PREMIUM" else calc_bj_sc
2193             py16.rectfill(186, 106, 50, 14, 0); py16.text("SCORE: " + str(score_val),
2194
2195             # Bei DOUBLE den effektiven Einsatz anzeigen
2196             if game_mode == "BLACKJACK" and bj_bets and bj_bets[0] != current_bet:
2197                 py16.rectfill(186, 120, 50, 8, 0)
2198                 py16.text("BET: $" + str(bj_bets[0]), 190, 122, 10)
2199
2200             if state == "GAME_OVER":
2201                 d_score = calc_score(dealer_hand) if game_mode == "PREMIUM" else calc_bj_s
2202                 py16.rectfill(186, 46, 50, 14, 0); py16.text("SCORE: " + str(d_score), 190
2203
2204             draw_chip(40, 110, current_bet, current_bet_color, False, 0)
2205
2206         # =====
2207         # --- OVERLAYS & UI (Immer ganz oben) ---
2208         # =====
2209
2210         for cx, cy, vx, vy in active_coins:
2211             py16.circfill(int(cx), int(cy), 3, 10); py16.circ(int(cx), int(cy), 3, 9)
2212
2213         # --- BETTING PHASE ---
2214         if state == "BETTING":
2215             if game_mode in ["SLOTS", "POKER"]:
2216                 options = ["10", "50", "100", "250", "INFO"]
2217                 for i, opt in enumerate(options):
2218                     bx = 13 + i * 46; by = 176; bw = 44; bh = 22
2219                     if i == bet_idx:
2220                         py16.rectfill(bx-2, by-2, bw+4, bh+4, 10); py16.rectfill(bx, by, bw, b
2221                     else:
2222                         py16.rectfill(bx, by, bw, bh, 5); py16.rectfill(bx+1, by+1, bw-2, bh-2
2223                     col = 7 if i != 4 else 11
2224                     py16.text(opt, bx + bw//2 - (len(opt)*4)//2 + 1, by + 9, col)
2225
2226             txt_play = "PRESS Z TO SPIN" if game_mode == "SLOTS" else "PRESS Z TO DEAL"
2227             py16.text(txt_play, 90, 208, 8)
2228             if bet_idx < 4 and money < bets[bet_idx]: py16.text("NOT ENOUGH MONEY!", 94, 1
2229
2230         elif game_mode == "ROULETTE":
2231             py16.text("SELECT YOUR BET:", 96, 106, 7)
2232             for i, b in enumerate(bets):
2233                 if i == 0: cx, cy = 32, 144
2234                 elif i == 1: cx, cy = 32, 192
2235                 elif i == 2: cx, cy = 224, 144
2236                 elif i == 3: cx, cy = 224, 192
2237                 if i == bet_idx: draw_chip(cx, cy, b, chip_colors[i], True, 2)
2238                 else: draw_chip(cx, cy, b, chip_colors[i], False, 0)
2239             py16.text("PRESS Z TO SELECT", 94, 116, 8)
2240             if money < bets[bet_idx]: py16.text("NOT ENOUGH!", 104, 126, 8)
```

PY16CASINO - CODE LISTING (PAGE 36)

```
2241
2242     else:
2243         py16.text("SELECT YOUR BET:", 80, 150, 7)
2244         for i, b in enumerate(bets):
2245             cx = 44 + i * 56; cy = 180
2246             if i == bet_idx: draw_chip(cx, cy, b, chip_colors[i], True, 2)
2247             else: draw_chip(cx, cy, b, chip_colors[i], False, 0)
2248         py16.text("PRESS Z TO DEAL", 90, 206, 8)
2249         if money < bets[bet_idx]: py16.text("NOT ENOUGH MONEY!", 94, 136, 8)
2250
2251 # --- ROULETTE BOARD PHASE (Cursor & Infos) ---
2252 elif state == "ROULETTE_BOARD":
2253     target = get_spot_from_cursor(roulette_cursor_x, roulette_cursor_y)
2254     if target is not None:
2255         cx, cy, cw, ch = get_target_rect(target)
2256         if cw > 0:
2257             py16.rect(cx, cy, cw, ch, 10); py16.rect(cx+1, cy+1, cw-2, ch-2, 10)
2258
2259         py16.text("Z:BET X:SPIN SPACE:CHIPS", 60, 106, 10)
2260         py16.text("TOTAL BET: {}/500".format(sum(roulette_bets.values()))), 84, 116, 11)
2261         draw_chip(20, 204, current_bet, current_bet_color, False, -4)
2262
2263 # --- ACTION MENUES ---
2264 elif state == "PLAYER_TURN" and game_mode in ["PREMIUM", "BLACKJACK"]:
2265     actions = bj_available_actions() if game_mode == "BLACKJACK" else ["HIT", "STAND"]
2266     n = len(actions)
2267     # Button-Breite je nach Anzahl: 2->54, 3->44, 4->38
2268     btn_w = 54 if n <= 2 else (44 if n == 3 else 38)
2269     gap = 6
2270     total_w = n * btn_w + (n - 1) * gap
2271     start = (256 - total_w) // 2
2272     by = 180
2273     for i, label in enumerate(actions):
2274         bx = start + i * (btn_w + gap)
2275         active = (i == action_idx)
2276         py16.rectfill(bx, by, btn_w, 20, 10 if active else 5)
2277         py16.rectfill(bx + 2, by + 2, btn_w - 4, 16, 0)
2278         tw = len(label) * 4
2279         py16.text(label, bx + btn_w // 2 - tw // 2 + 1, by + 8, 10 if active else 6)
2280     py16.text("SELECT: < >   CONFIRM: Z", 64, 208, 7)
2281
2282 # --- POPUPS ---
2283 elif state == "GAME_OVER":
2284     if game_mode not in ["SLOTS", "POKER"]:
2285         box_y = 100 if game_mode == "ROULETTE" else 140
2286         py16.rectfill(50, box_y, 156, 44, 0); py16.rect(50, box_y, 156, 44, 5)
2287         msg_x = 128 - (len(msg) * 4) // 2
2288         msg2_x = 128 - (len(msg2) * 4) // 2
2289
2290         py16.text(msg, msg_x, box_y + 10, 10); py16.text(msg2, msg2_x, box_y + 26, 7)
2291         if (py16.t() // 30) % 2 == 0: py16.text("PRESS Z TO CONTINUE", 90, box_y + 50,
2292         else:
2293             if (py16.t() // 30) % 2 == 0: py16.text("PRESS Z TO CONTINUE", 90, 166, 8)
2294
2295 # --- Achievement-Toast immer ganz oben rendern ---
2296 draw_achievement_toast()
2297
2298 if __name__ == "__main__":
2299     py16.run(update, draw, init)
2300
```